

Test und Zuverlässigkeit

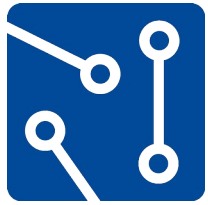
Test and Reliability

Summer Term 2026

Prof. Dr. Ralf Wimmer

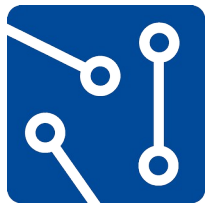
Dr. Tobias Seufert

Chapter 3: Test Generation for Single Stuck-At Faults in Combinational Circuits



Test Generation Problem

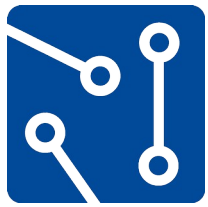
- Given a set of **single stuck-at faults** F
 - Find a set of tests T such that, **for every detectable fault in F** , there is a test in T that detects it.
- Consideration of **further** requirements
 - Energy usage (→ often higher switching activity during test than normal)
 - Compactness of test set (→ time for applying tests)
 - Compatibility with test environment



Types of Test Generation Procedures

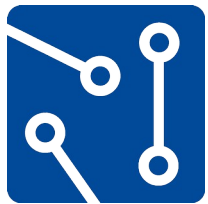
- **Random** test generation procedures produce random tests that are independent of the circuit structure.
- **Deterministic** test generation procedures produce tests by processing a model of the circuit.

ATPG = Automatic Test Pattern Generation



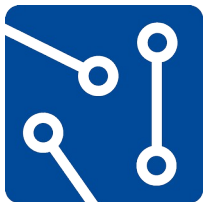
Chapter Outline

- Pre-ATPG patterns ([pseudo-] random)
- Deterministic ATPG: D-Algorithm
- Test compaction
 - Static
 - Dynamic
- Power-aware test generation
- Deterministic ATPG: SAT-based (next chapter)

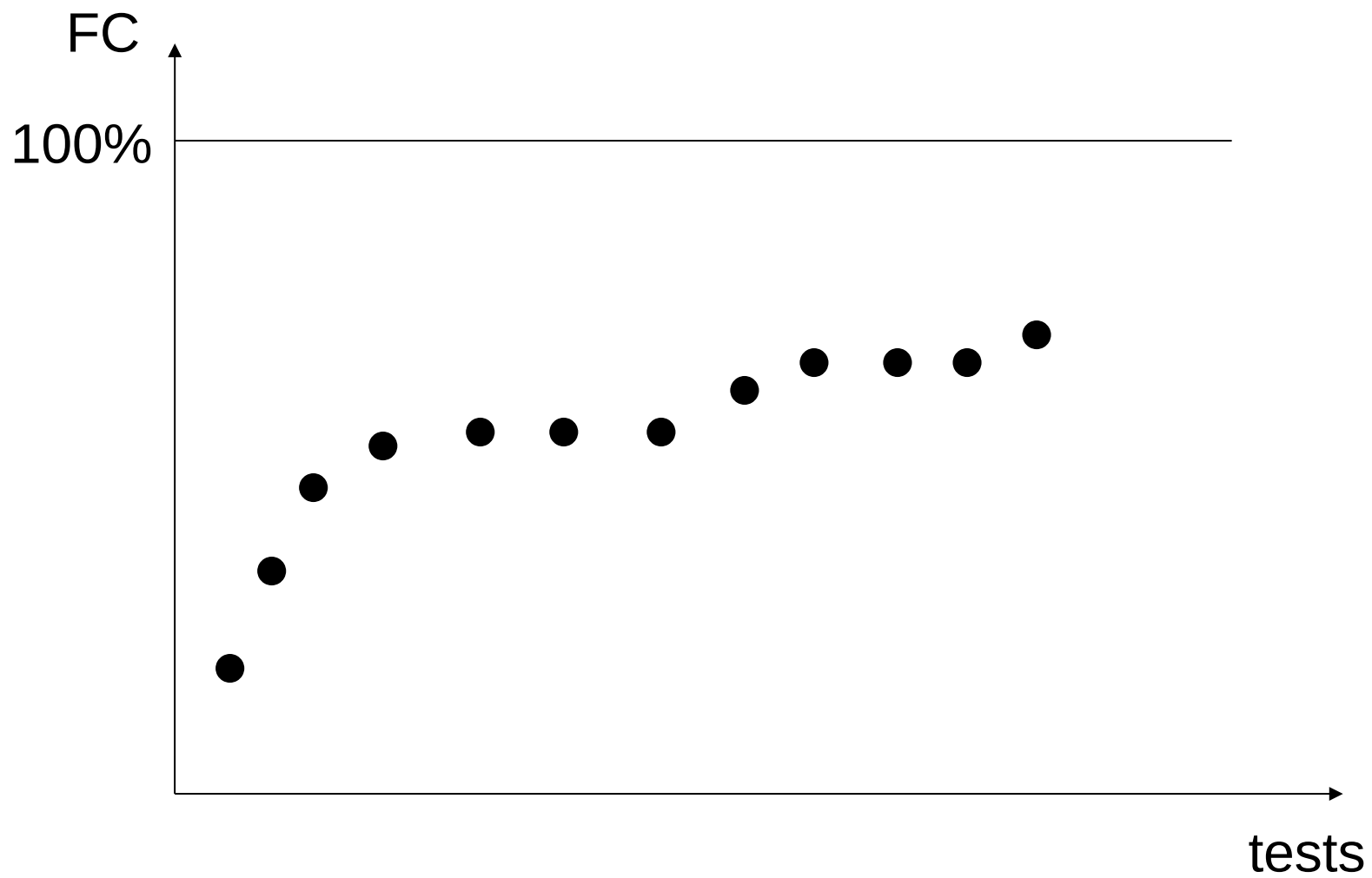


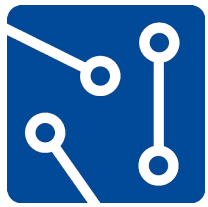
Random Patterns

- Let F be the set of target faults.
Set $n_{\text{ineffective}} := 0$ and $T := \{\}$.
- Generate a **random pattern** t
- **Fault simulate** t .
Let the set of detected faults be F_{det} .
- If $|F_{\text{det}}|$ is below threshold:
 Set $n_{\text{ineffective}} := n_{\text{ineffective}} + 1$.
Else:
 Remove F_{det} from F , add t to T and set $n_{\text{ineffective}} := 0$.
- If $n_{\text{ineffective}}$ **reached an upper bound**, stop, else repeat.



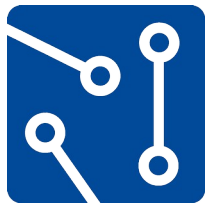
Fault Coverage Curve





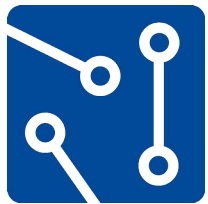
Fault Simulation Process

- Generate random patterns one at a time:
 - **Parallel fault simulation** (or deductive or event-based fault simulation)
- Generate W random patterns at once:
 - **Parallel pattern simulation** possible.



Simulation-Based Test Generation

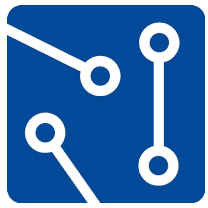
- Produce one or more test **candidates**.
 - Modify them based on certain parameters until target faults are detected.
 - Types of simulation-based procedures:
 - Simple random test generation
 - Procedures based on, e.g.,
 - Genetic optimization or simulated annealing
 - Properties of the circuit and its test patterns
- Example: **Sensitive Patterns**



Sensitive Patterns

- A change in the value of a sensitive input causes a change in the value of a sensitive output.
- It is *likely* that the **presence of a fault** on a path from a sensitive input to a sensitive output will have the same effect as changing the input value, and the **fault will be detected**.

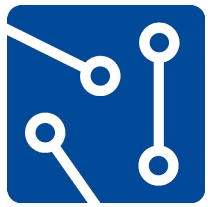
Incomplete in case of reconvergences!



Example

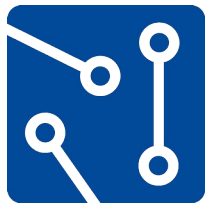
(circuit with inputs A_1, A_2, A_3, A_4 and outputs O_1, O_2)

$A_1A_2 =$		00	01	11	10
$A_3A_4 =$	00	11	11 S	11	11
	01	11	11 S	11	11 S
	11	11 S	00 S	11 S	10 S
	10	11 S	01 S	11 S	11 S



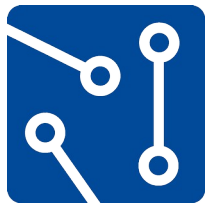
Types of Deterministic Procedures

- **Branch-and-bound** procedures
 - Complete, can also prove redundancy of faults.
 - Exponential complexity (The problem itself is coNP-complete!)
- **Simulation-based** procedures
 - Incomplete
 - Linear complexity
- Fault-oriented procedures
- Fault-independent procedures



Three-Valued Logic (01X-Logic)

- **Unspecified/unknown** values (U or X) are necessary for:
 - **Incompletely** specified circuit logic
 - Handling **uncontrollable** inputs or state bits
 - **Abstract** from unknown faulty behavior
- Usually accommodated by representing each logic value **using two bits**



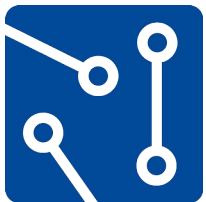
Three-Value Logic: Encoding in Two-Value Logic

	Value of A		
	0	1	U/X
A_1 / A_2	0 / 0	1 / 1	0 / 1

$C = A \cdot B$ is implemented as $C_1 = A_1 \cdot B_1$, $C_2 = A_2 \cdot B_2$

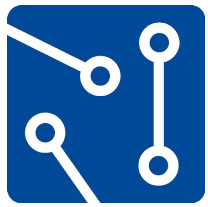
$B = A'$ is implemented as $B_1 = A_2'$, $B_2 = A_1'$

$C = (A \cdot B)'$ is implemented as $C_1 = (A_2 \cdot B_2)'$, $C_2 = (A_1 \cdot B_1)'$



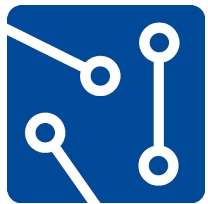
Example: AND-Gate

	<i>A</i>		
<i>B</i>	00 (=0)	11 (=1)	01 (=X)
00 (=0)	00	00	00
11 (=1)	00	11	01
01 (=X)	00	01	01



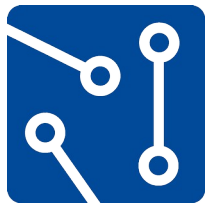
Fault-Oriented Test Generation (D-Algorithm, Roth 1966)

- Overall process (I):
 - Select a fault f from F .
 - Generate a test t for f .
 - If f is redundant, remove f from F .
 - **Simulate** all the faults in F under t .
Remove from F all the **detected faults**
 - → **Fault dropping**.
 - Repeat until F is empty.



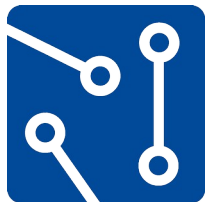
Fault-Oriented Test Generation

- To generate a test for a fault $l\ s@v$, it is necessary to:
 - **Activate** the fault by setting $l = v'$.
 - **Propagate** the fault effect to a primary output.
- Value system:
 - For activation three values $\{0, 1, x\}$.
 - For propagation five values $\{0, 1, 0/1, 1/0, x\}$.
 - Abbreviations: $D = 1/0$, $D' = 0/1$ (different values in faulty and fault-free case)
 - $\rightarrow$ Values $\{0, 1, D, D', x\}$



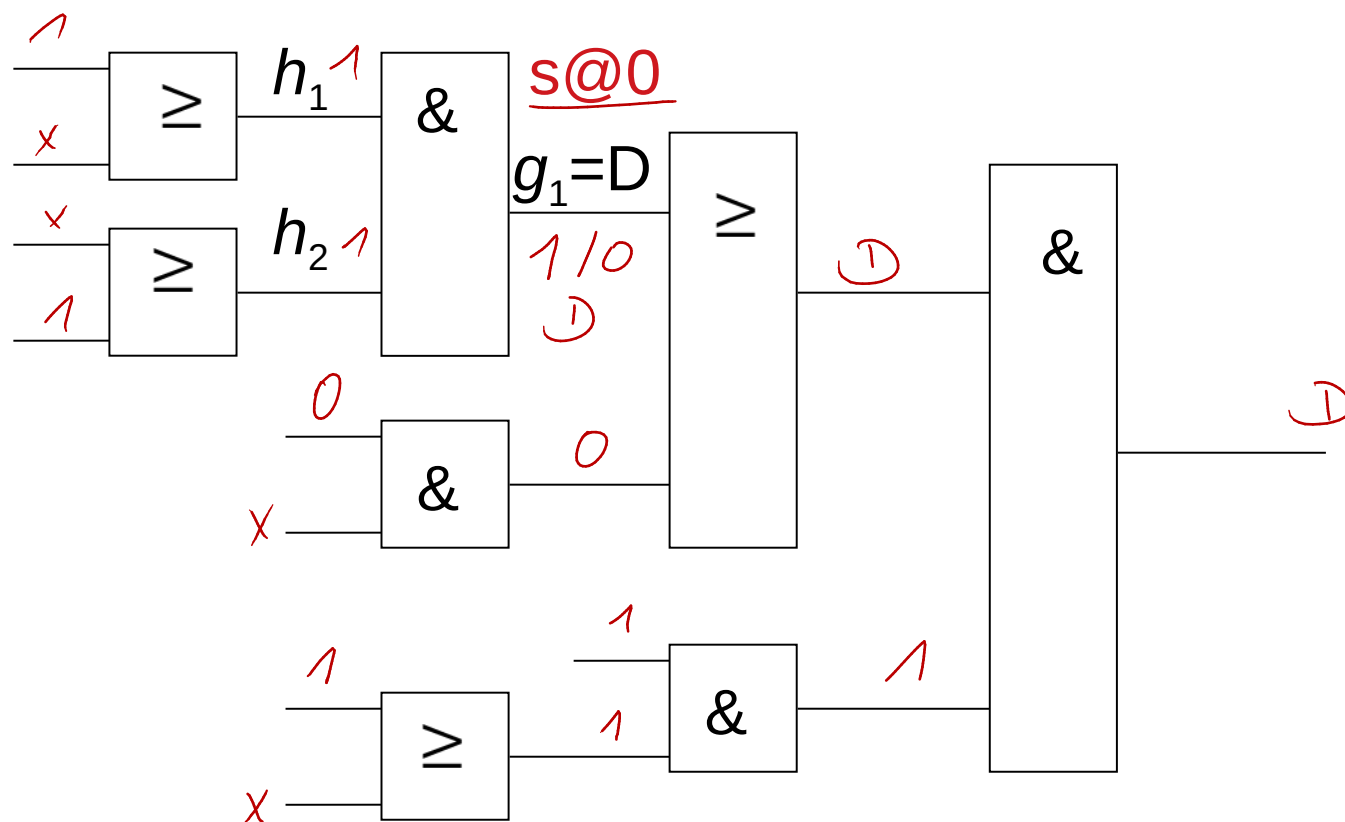
ATPG Tasks

- **Fault activation** requires to solve a line justification problem:
 - Set l to v'
 - $\text{Justify}(l, v')$
- **Fault propagation** ($\text{Propagate}(l, D)$ or $\text{Propagate}(l, D')$) can be transformed into a set of line justification problems by selecting a path along which the fault effect will be propagated.

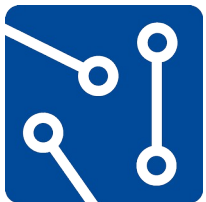


Example: Fan-out Free Circuit

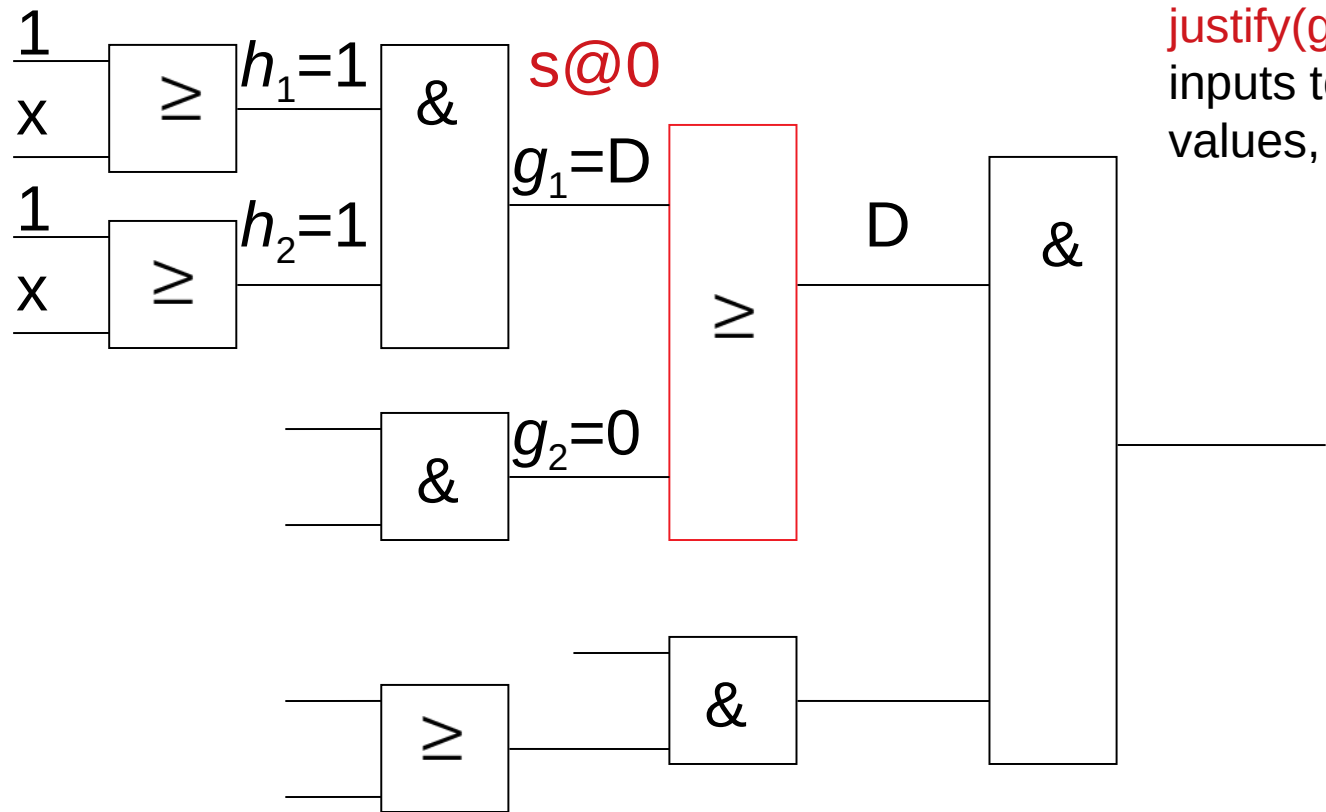
Justify($g_1, 1$)



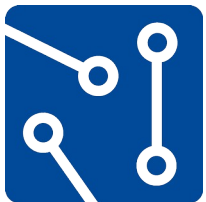
Justify($g_1, 1$)
is reduced to solving
Justify($h_1, 1$)
and
Justify($h_2, 1$)



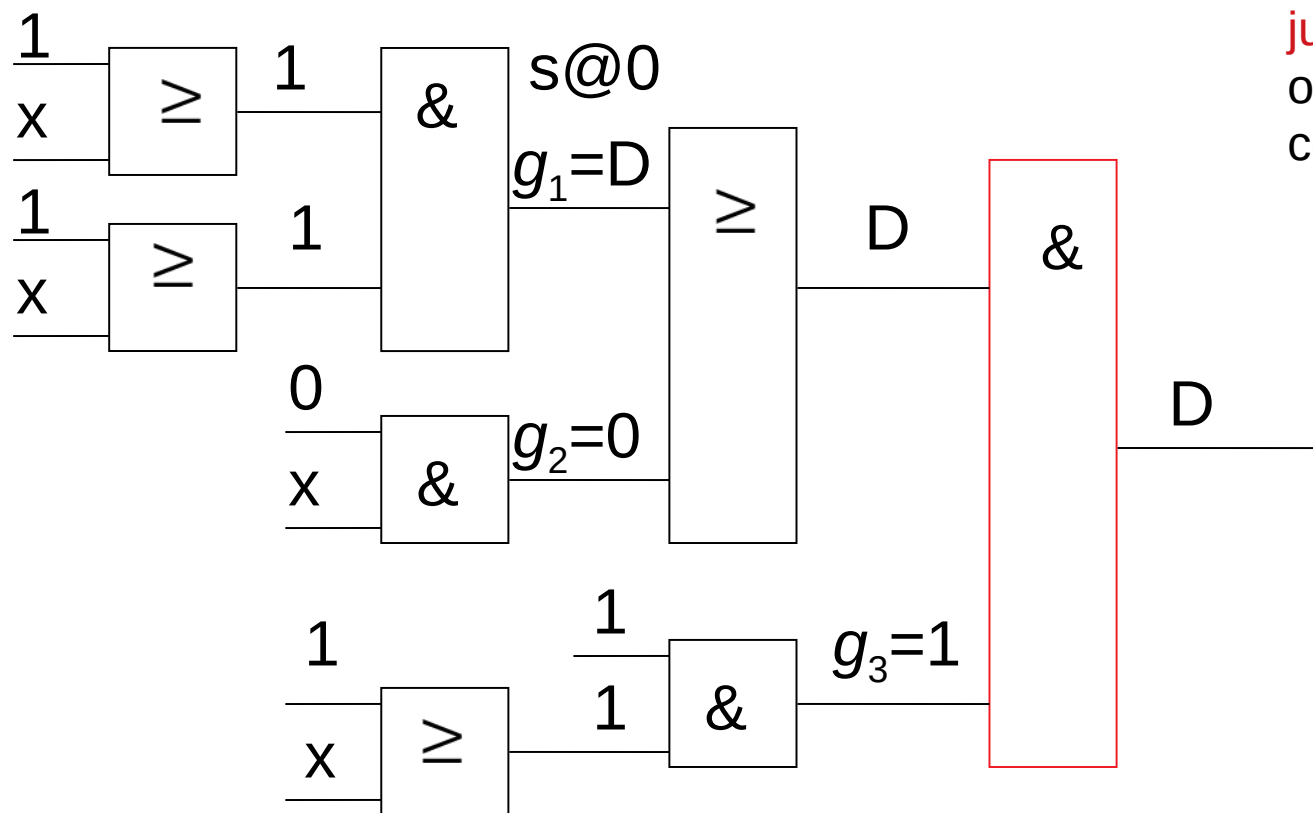
Justify($g_2, 0$)



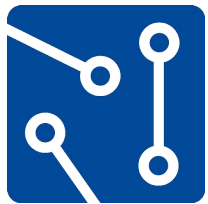
Propagation of the fault effect over the OR-gate requires **justify($g_2, 0$)** (= setting the other inputs to the non-controlling values, here: 0).



Justify($g_3, 1$)

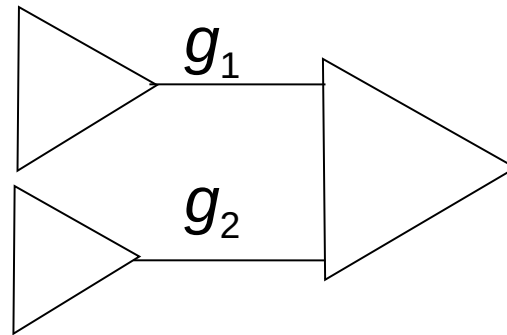


Propagation of the fault effect over the AND-gate requires **justify($g_3, 1$)** (= setting the other inputs to the non-controlling values, here: 1).

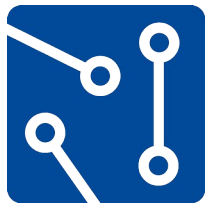


Fan-out Free Circuit Properties

- There is **only one path** for propagating a fault effect to a primary output.
- If g_1 does not drive g_2 , the sets of inputs driving g_1 and g_2 are disjoint.

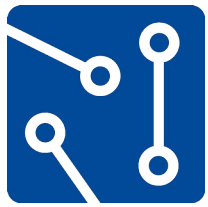


- Each line justification problem can be solved **independently** of all others.

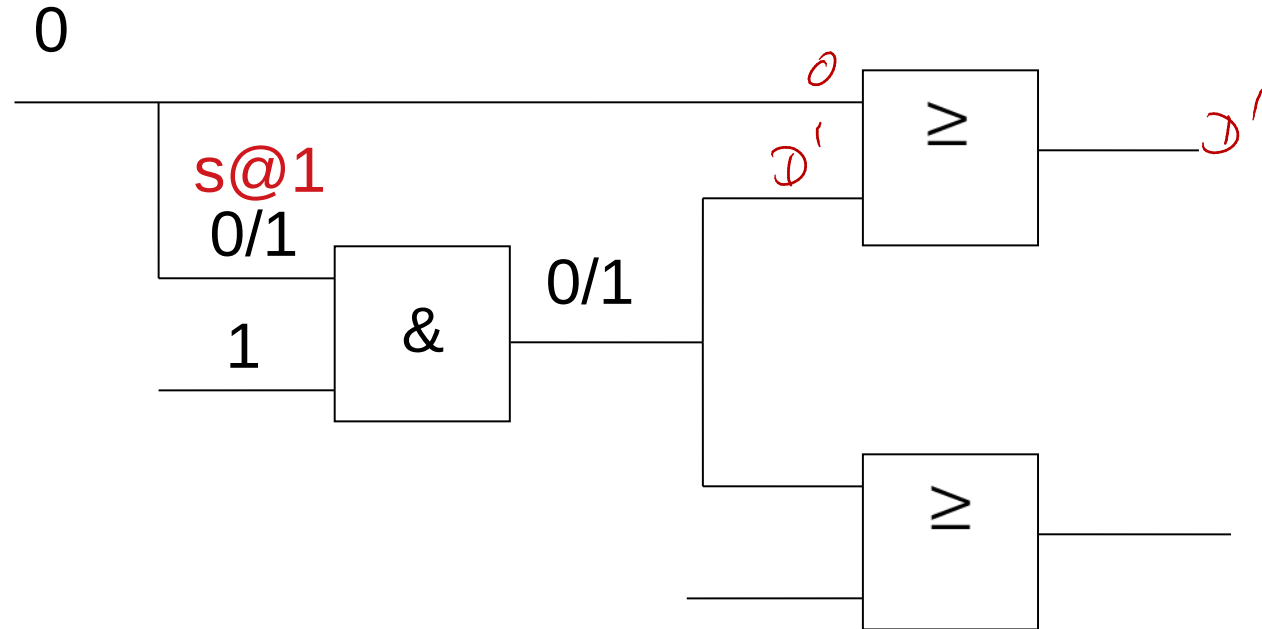


Circuits with Fan-out

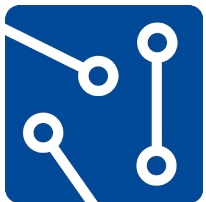
- We may have **more than one path** for propagating a fault effect to a primary output.
- **Line justification** for two different lines may result in **conflicts** if the two lines are considered separately.
- Line justification can fail, e.g., because a fault is **undetectable**.



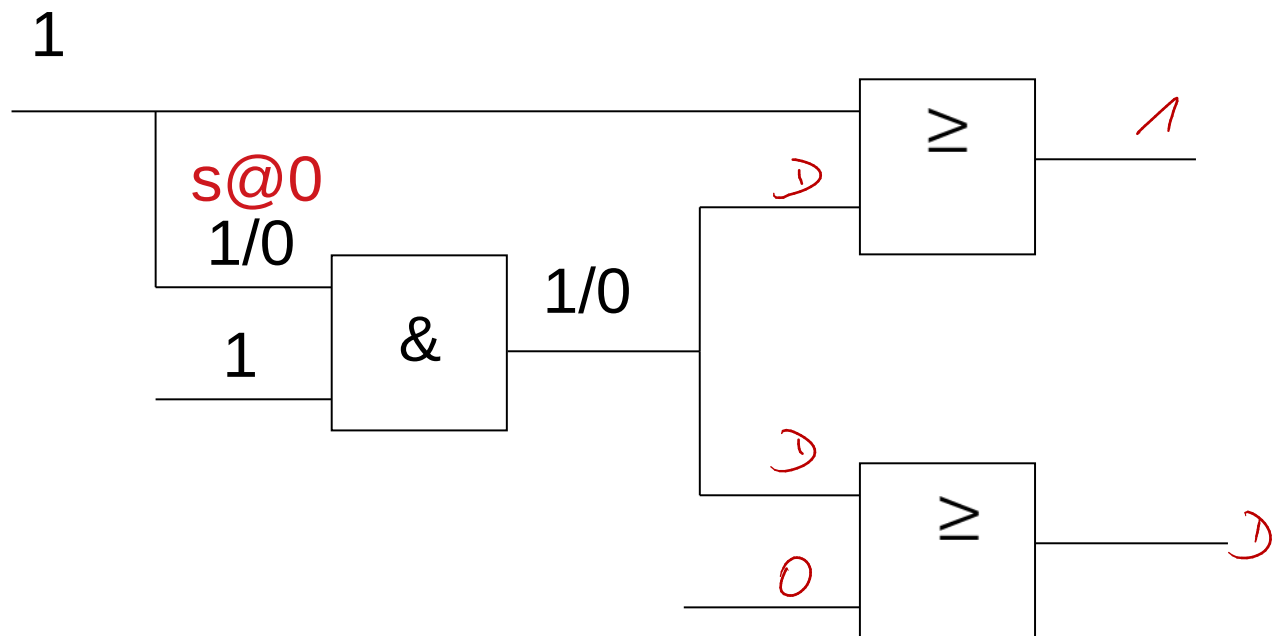
Example 1



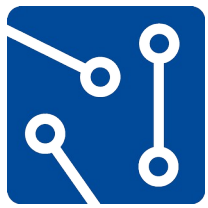
We can choose one of two paths for propagating the fault effect to an output



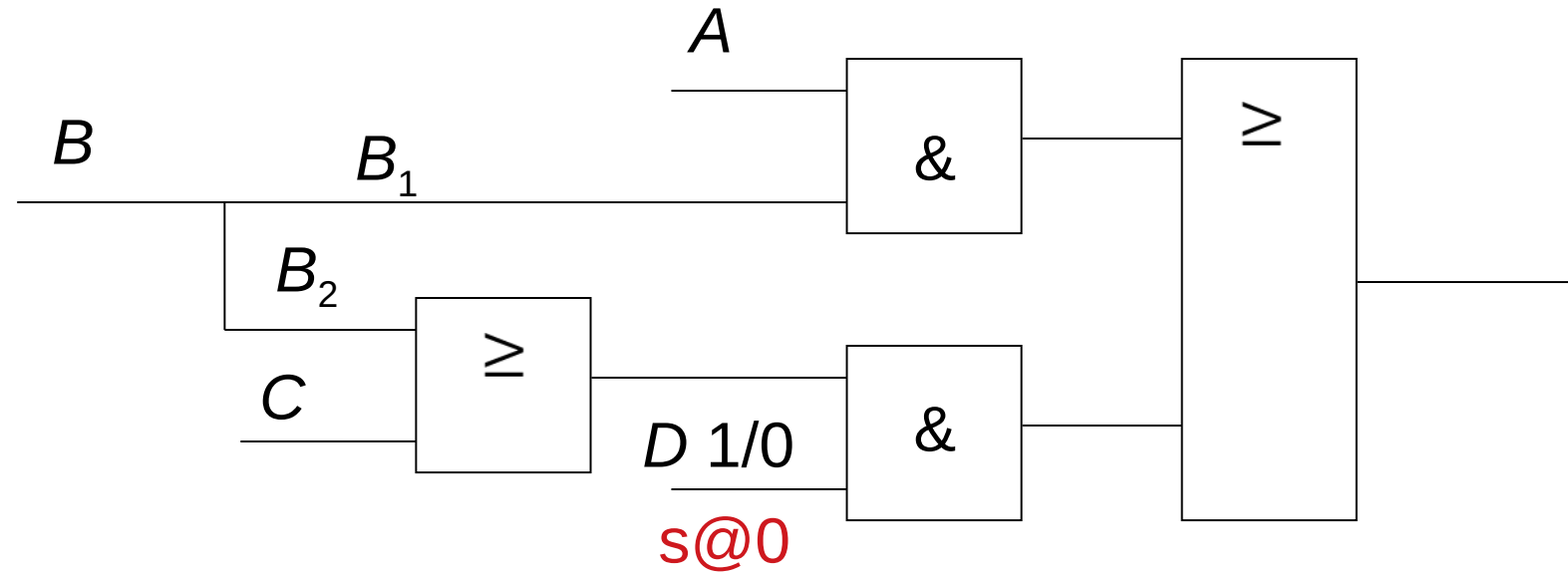
Example 2



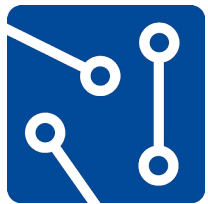
We can choose one of two paths for propagating the fault effect to an output; one of the paths will not result in a test.



Example 3

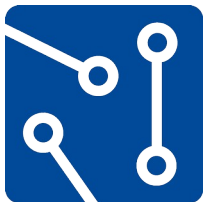




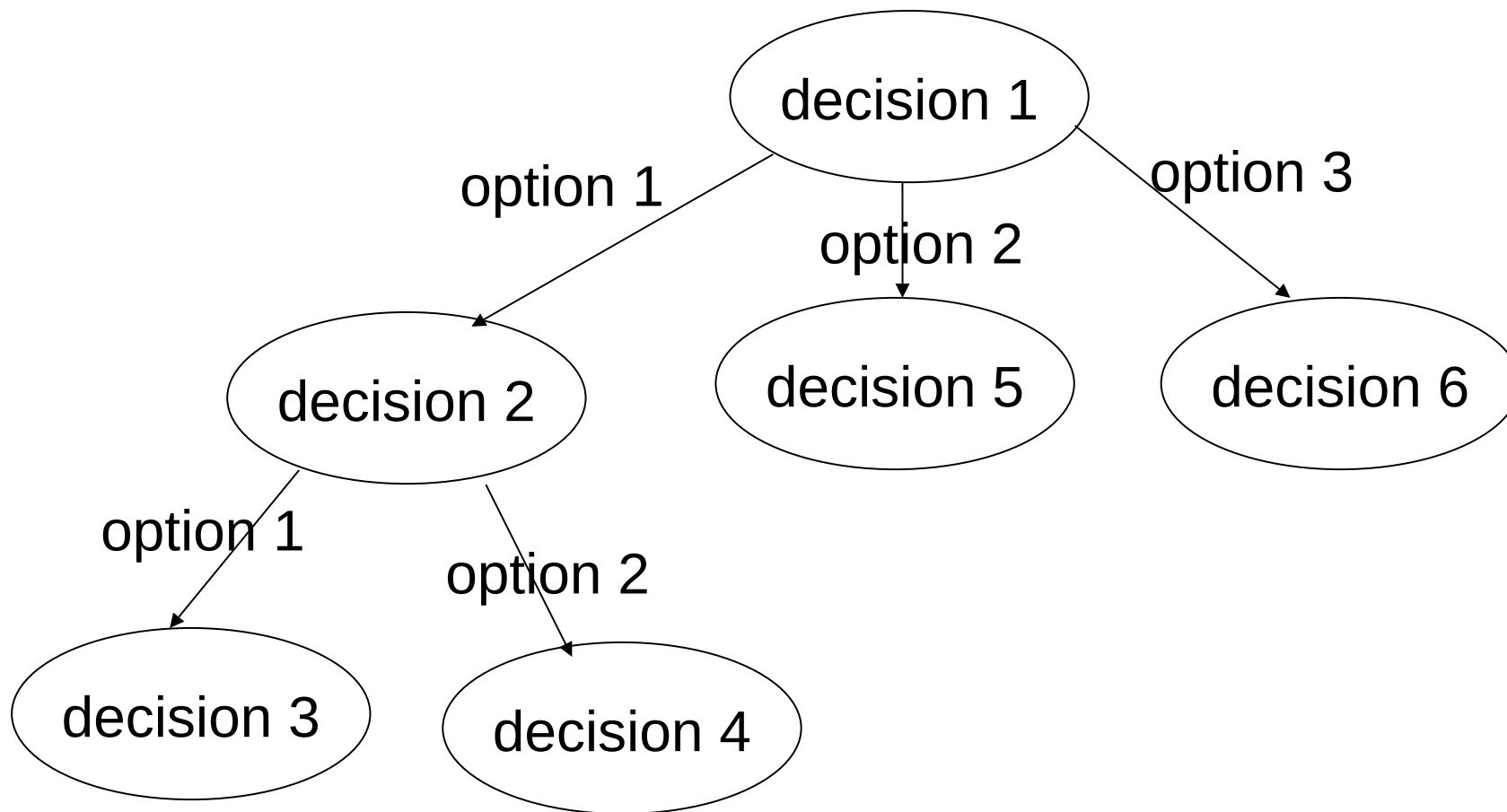


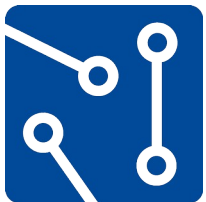
Backtracking

- The search for a test requires that we **make decisions**.
 - Select a path for fault effect propagation.
 - Justification requires a controlling value on a single gate input.
- A decision may lead to a **conflict**.
- **Backtracking** consists of **undoing** the last decision where a different choice exists, and making a **different decision**.

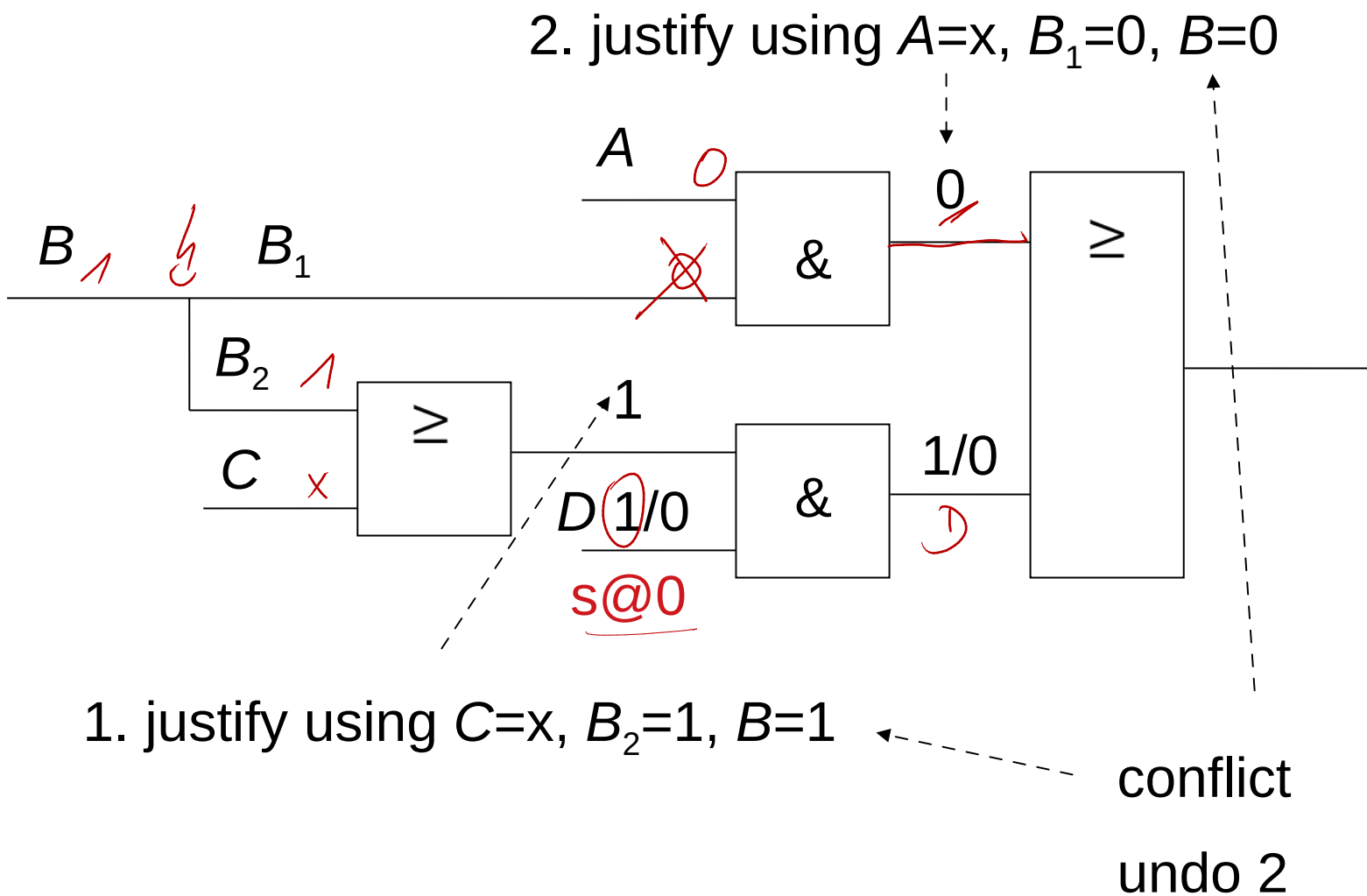


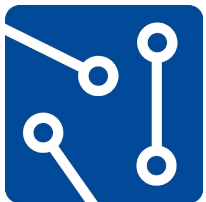
Branch-and-Bound Process



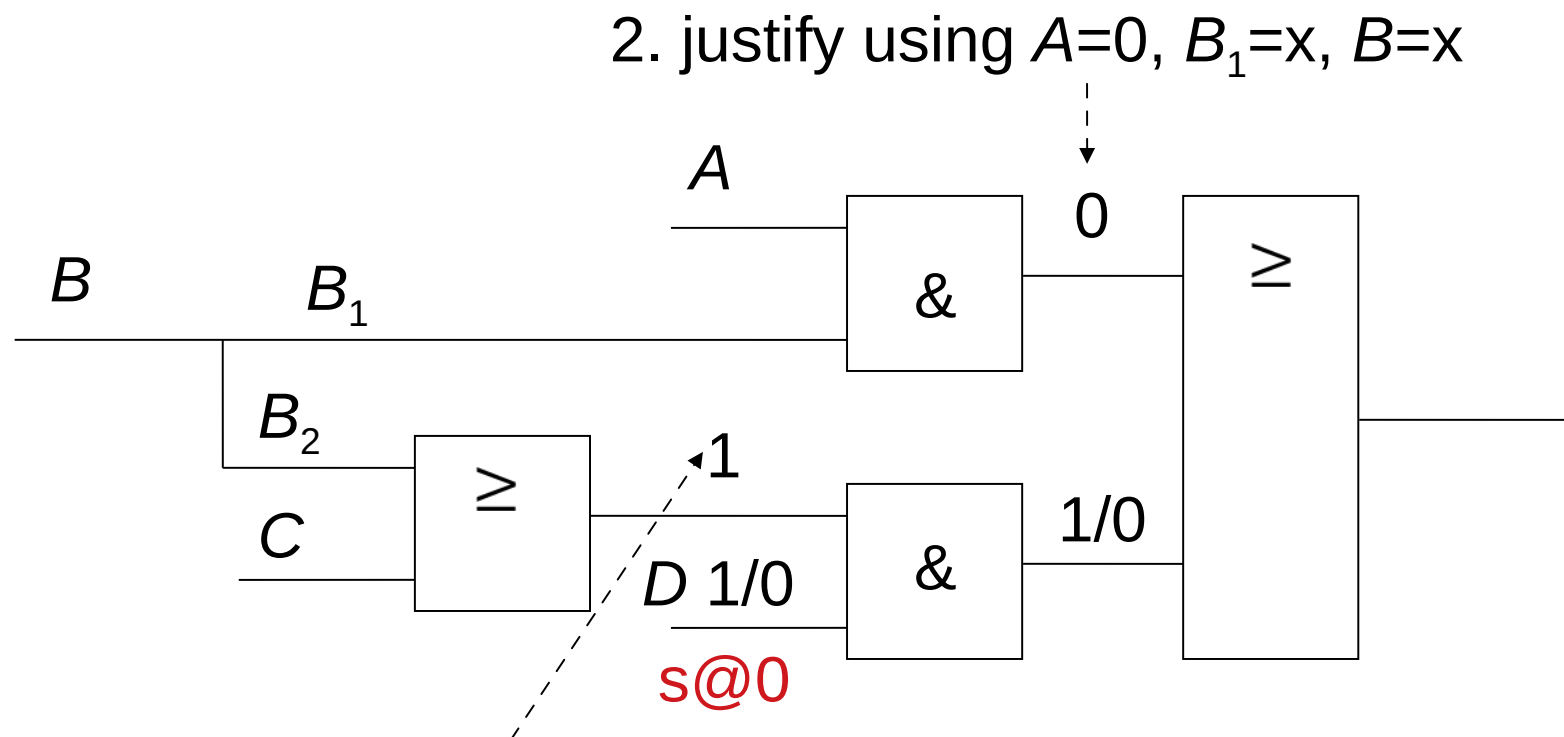


Example 3 with Backtracking



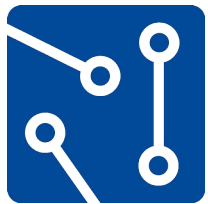


Example 3 with Backtracking



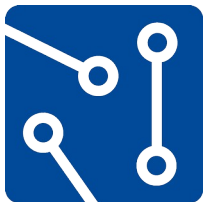
1. justify using $C=x, B_2=1, B=1$

test: $A=0, B=1, D=1$

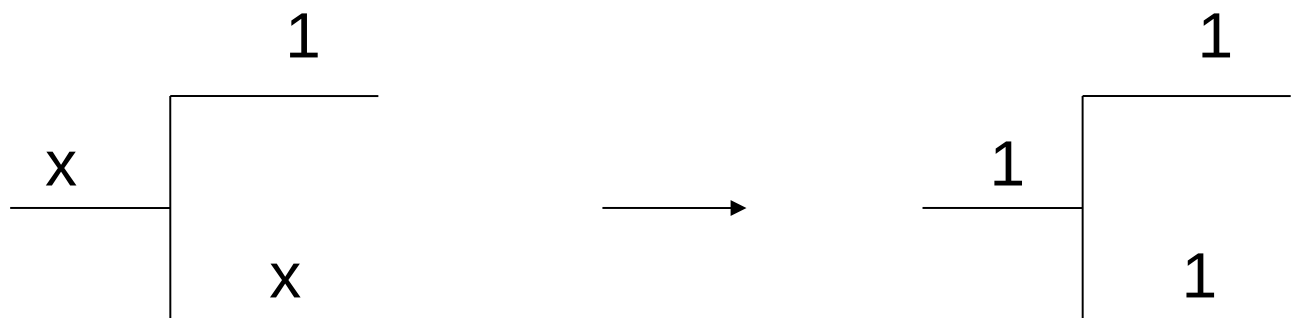
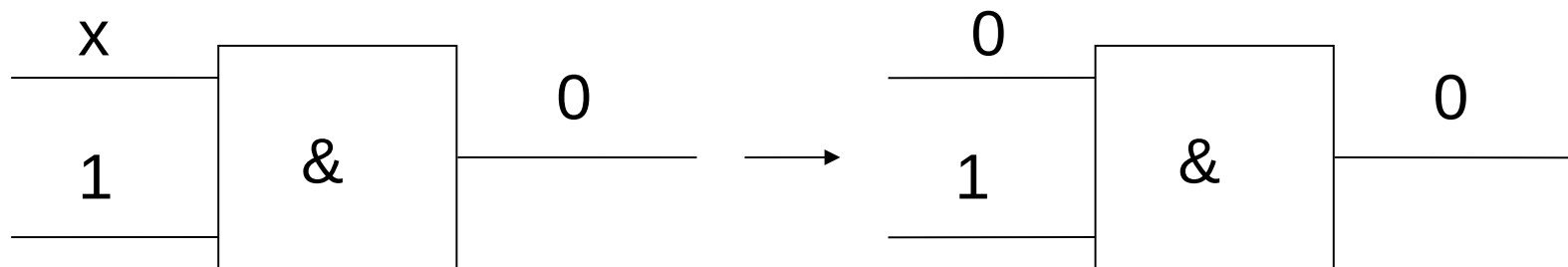
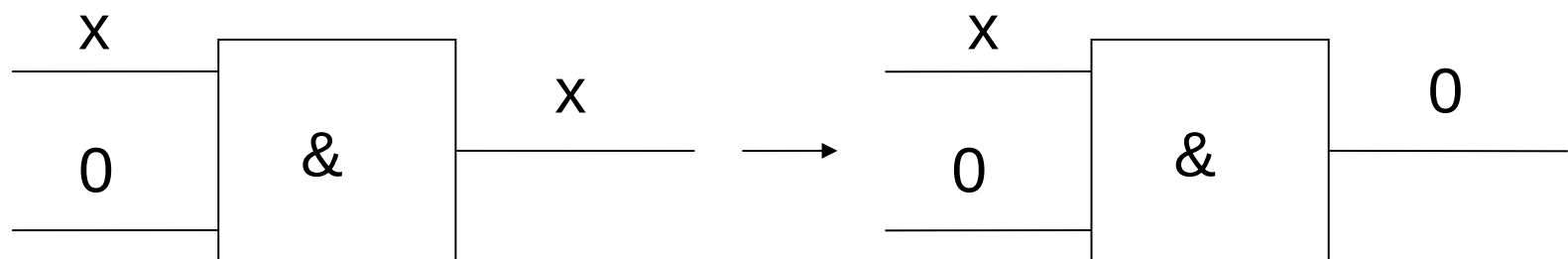


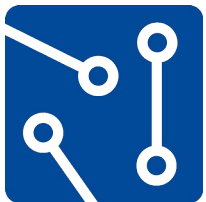
Implications

- For every value assigned, find all the values implied by it.
- Some **decisions** (and conflicts) can be avoided by finding implications.
- For some faults, **decisions** can be **avoided** altogether by using implications
- „If you can know what to do, don't guess.“



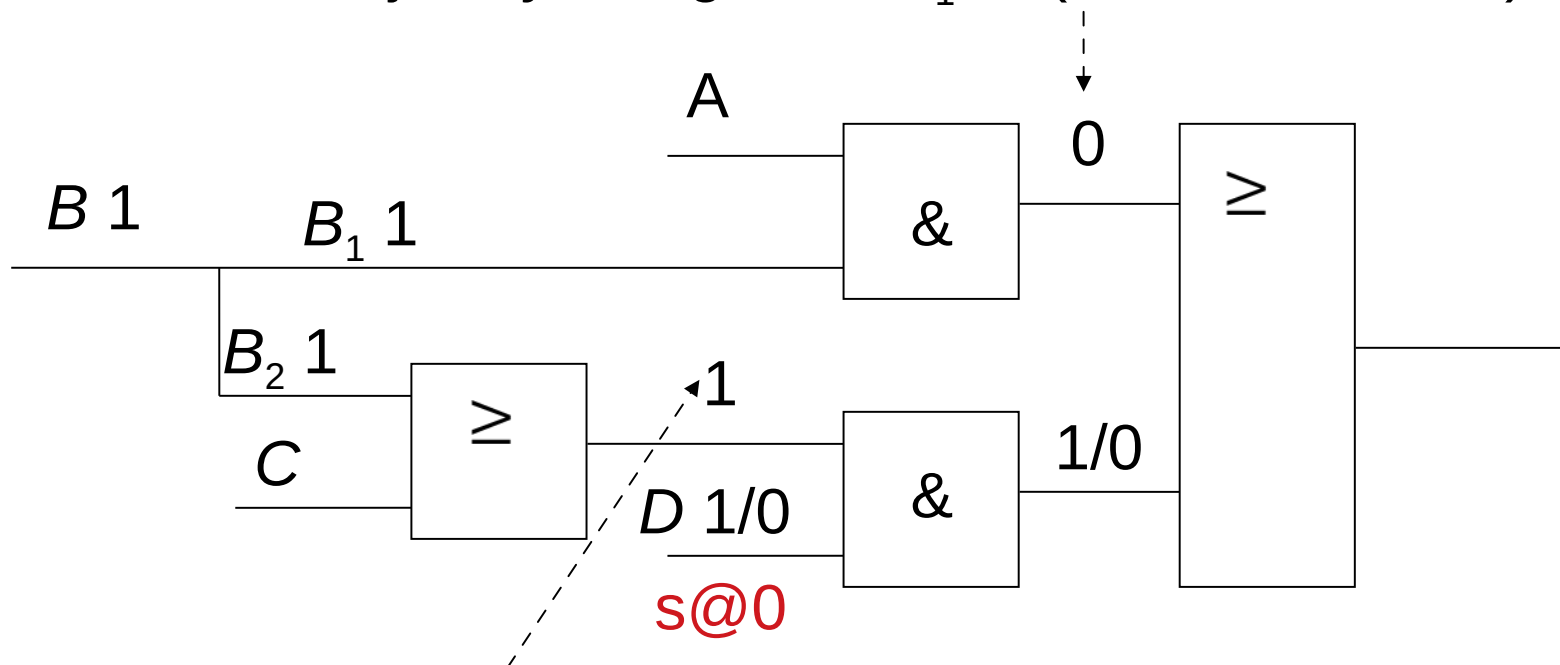
Examples of (Local) Implications





Example 3 with Implications

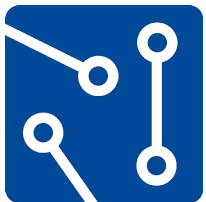
2. justify using $A=0, B_1=1$ (no other choice)



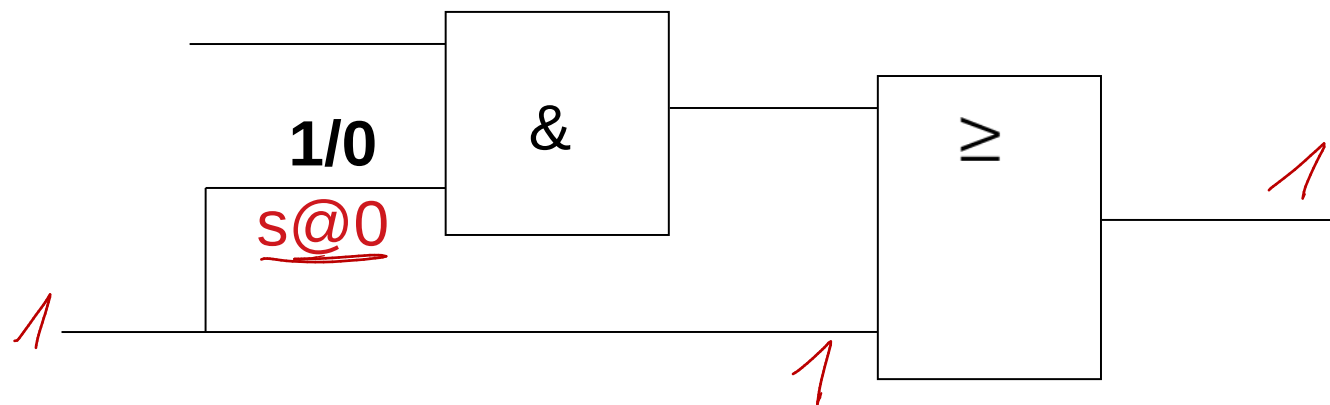
1. justify using $C=x, B_2=1$.

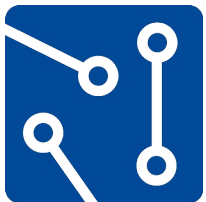
implications: $B=1, B_1=1$

test: $A=0, B=1, D=1$

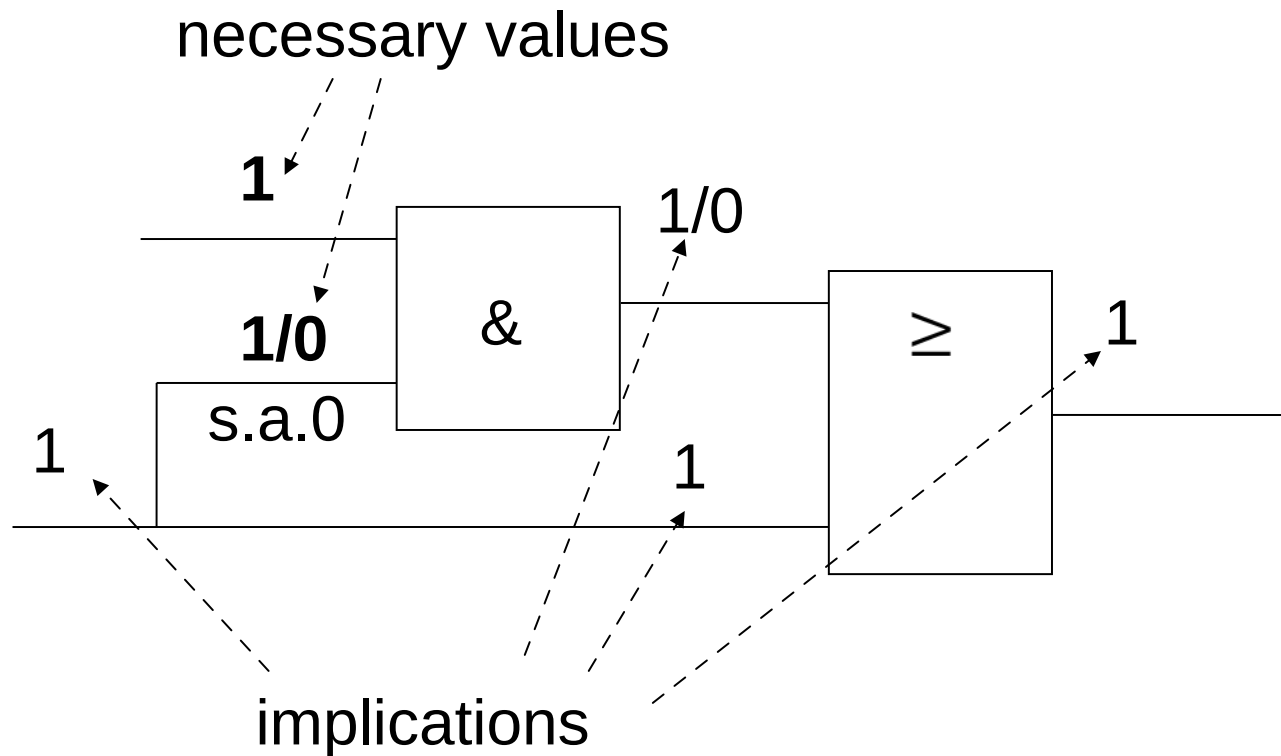


Example 4

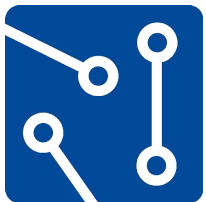




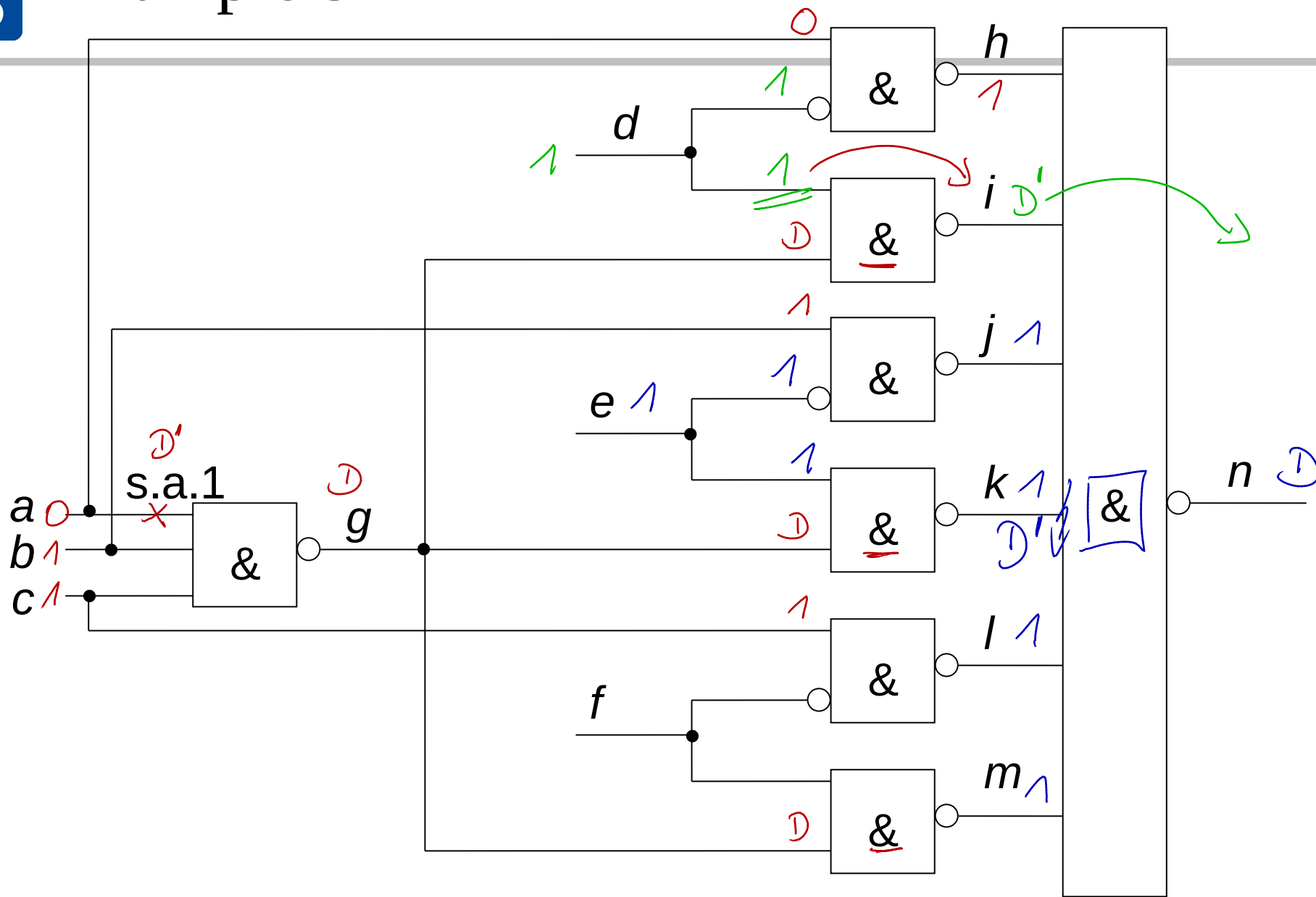
Example 4

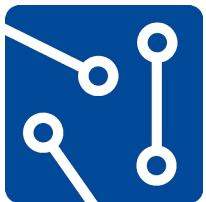


The fault is undetectable



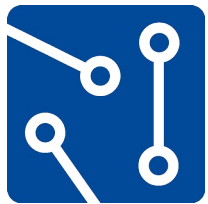
Example 5





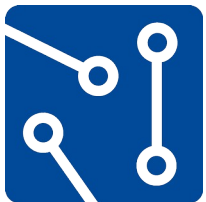
Value Computation in Example 5

Decisions	Implications	Notes
	$\underline{a} = 0, g = D$ $b = c = h = 1$	Fault activation Propagation must go through g
$d = 1$	$i = D'$	Propagate through i
$j = k = l = m = 1$	$n = D, e = 1, k = D'$	Propagate through n Contradiction
$\underline{e} = 1$	$k = D', j = 1$	Propagate through k
$l = m = 1$	$n = D, f = 1, m = D'$	Propagate through n Contradiction
$f = 1$	$m = D', l = 1, n = D$	Propagate through m



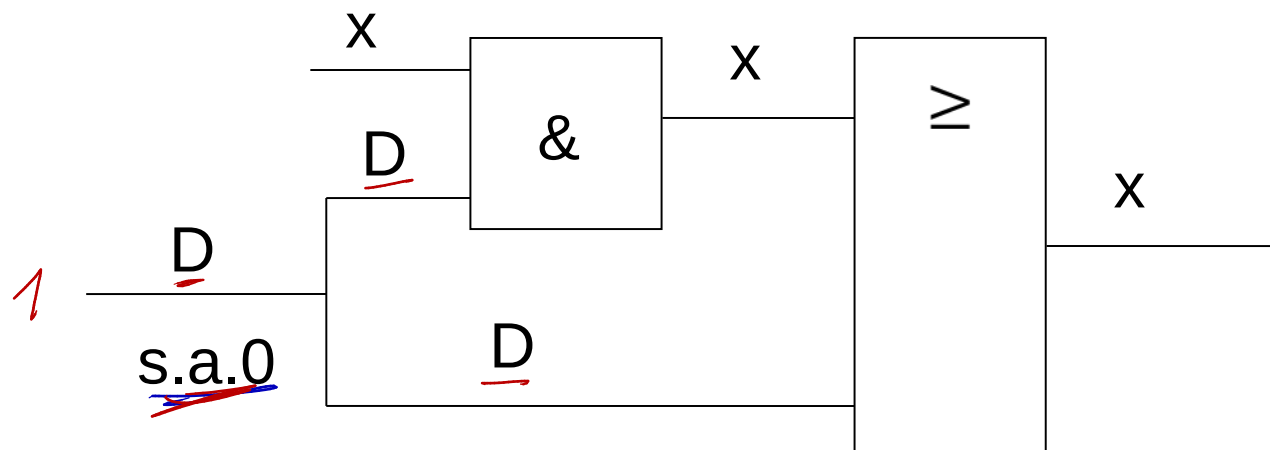
The 9-V Logic

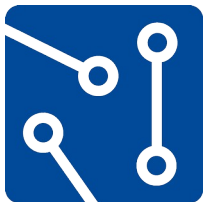
- So far, we used the logic values $\{0, 1, D, D', x\}$.
- The 9-V logic takes into account that it is possible to have a **known value** in the **fault free** (faulty) circuit while the value in the **faulty** (fault free) circuit is **unknown**.
- The value system of the 9-V logic includes
$$\{0, 1, D, D', 0/x, 1/x, x/0, x/1, x\}.$$
- $0/x$ can be $0/0 (= 0)$ or $0/1 (= D')$, and so on.



Example – 5 Values

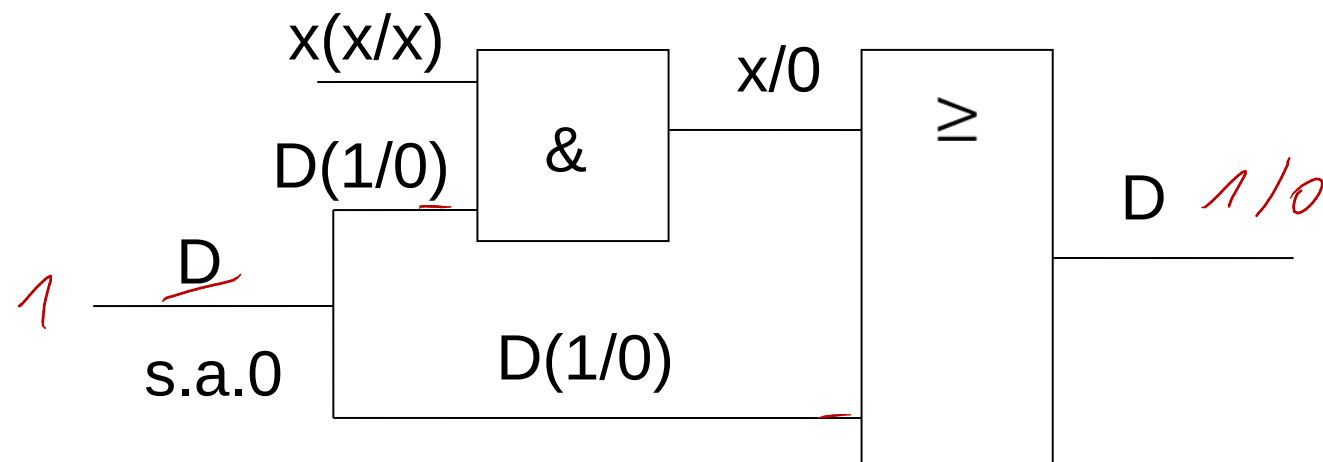
Initial assignment has no implications

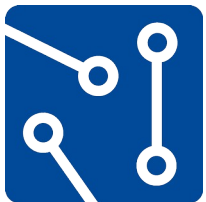




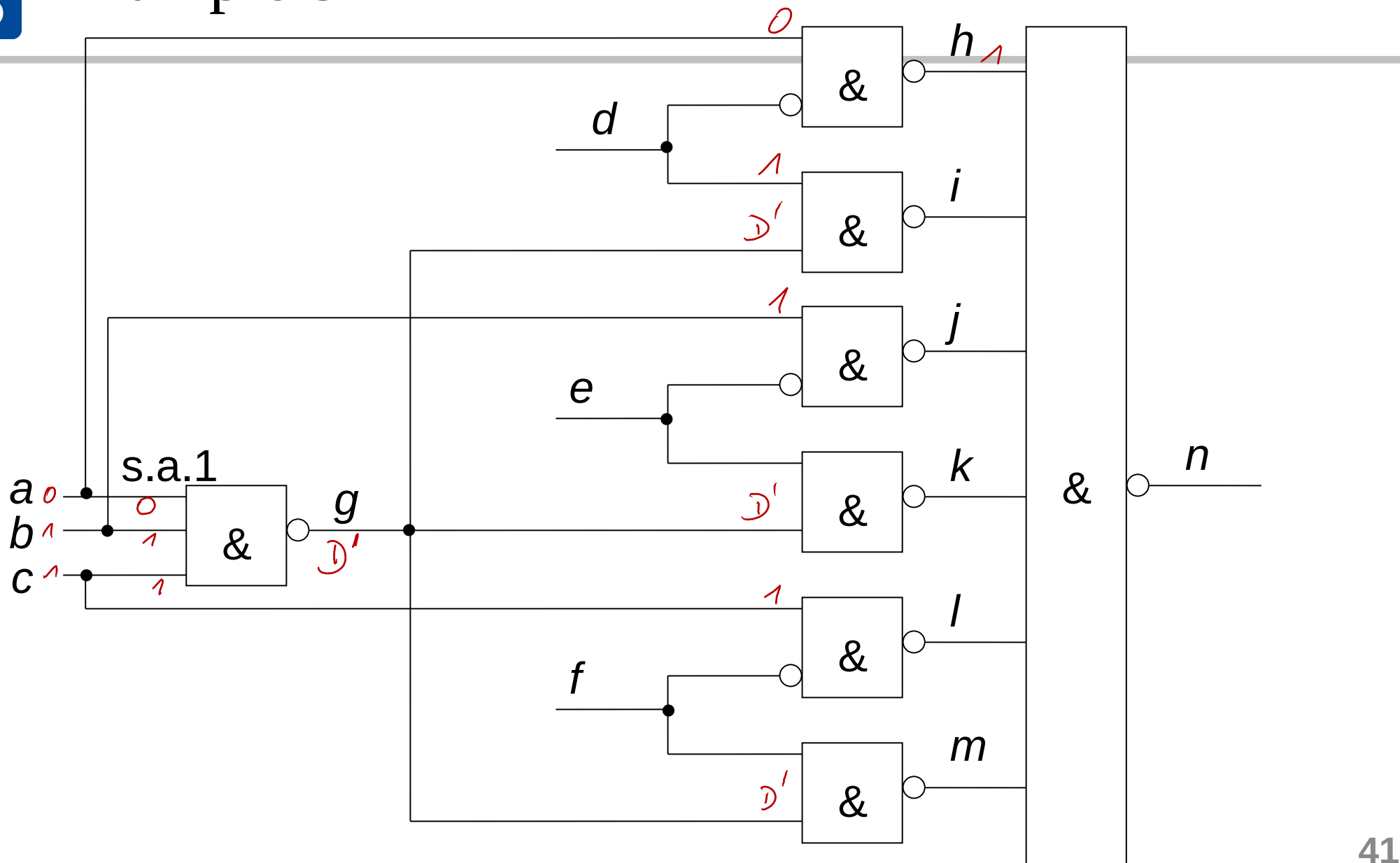
Example – 9 Values

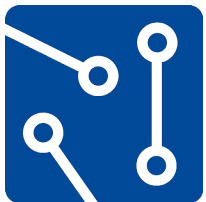
Initial assignment is sufficient to detect the fault





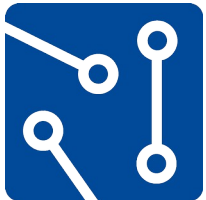
Example 5





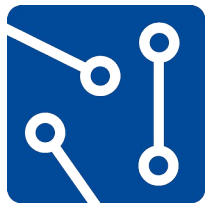
5V Computation in Example 5

Decisions	Implications	Notes
	$a = 0, g = D$ $b = c = h = 1$ ✓	Fault activation Propagation must go through g
$d = 1$	$i = D'$	Propagate through i
$j = k = l = m = 1$	$n = D, e = 1, f = 1,$ <u>$k = D', m = D'$</u>	Propagate through n Conflict
$e = 1$	$k = D', j = 1$	Propagate through k
$l = m = 1$	$n = D, f = 1,$ $m = D'$	Propagate through n Conflict
$f = 1$	$m = D', l = 1, n = D$	Propagate through m



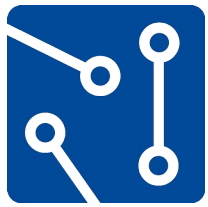
9V Computation in Example 5

Decisions	Implications	Notes
	$a = 0, g = D$ $b = c = h = 1$ $i = k = m = x/1$	Fault activation Propagation must go through g More implications
$d = 1$	$i = D', n = \underline{1/x}$	Propagate through i
$j = l = x/1$	$n = D, e = f = 1,$ $k = m = D'$	Propagate through n



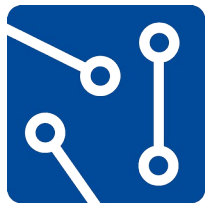
ATPG Procedure

- Consists of:
 - Initial necessary values for activation and propagation.
 - Implications
 - Decisions related to line justification
 - Decisions related to propagation
 - Backtracking



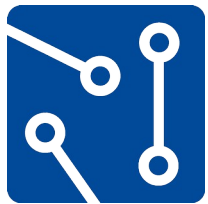
Outline of an ATPG Procedure

- Solve()
 - if imply() = FAIL return FAIL
 - If (error at PO and all lines are justified) return SUCCESS
 - If (no error can be propagated to a PO) return FAIL
 - select an unsolved problem
 - repeat
 - ▶ Select an untried way to solve it
 - ▶ If Solve() = SUCCESS return SUCCESS
 - until all ways to solve it have been tried
 - return FAIL



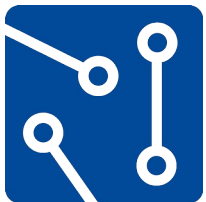
Data Structures used by the D-Algorithm

- Decision tree (for backtracking)
- D-frontier (for propagation)
- J-frontier (for line justification)

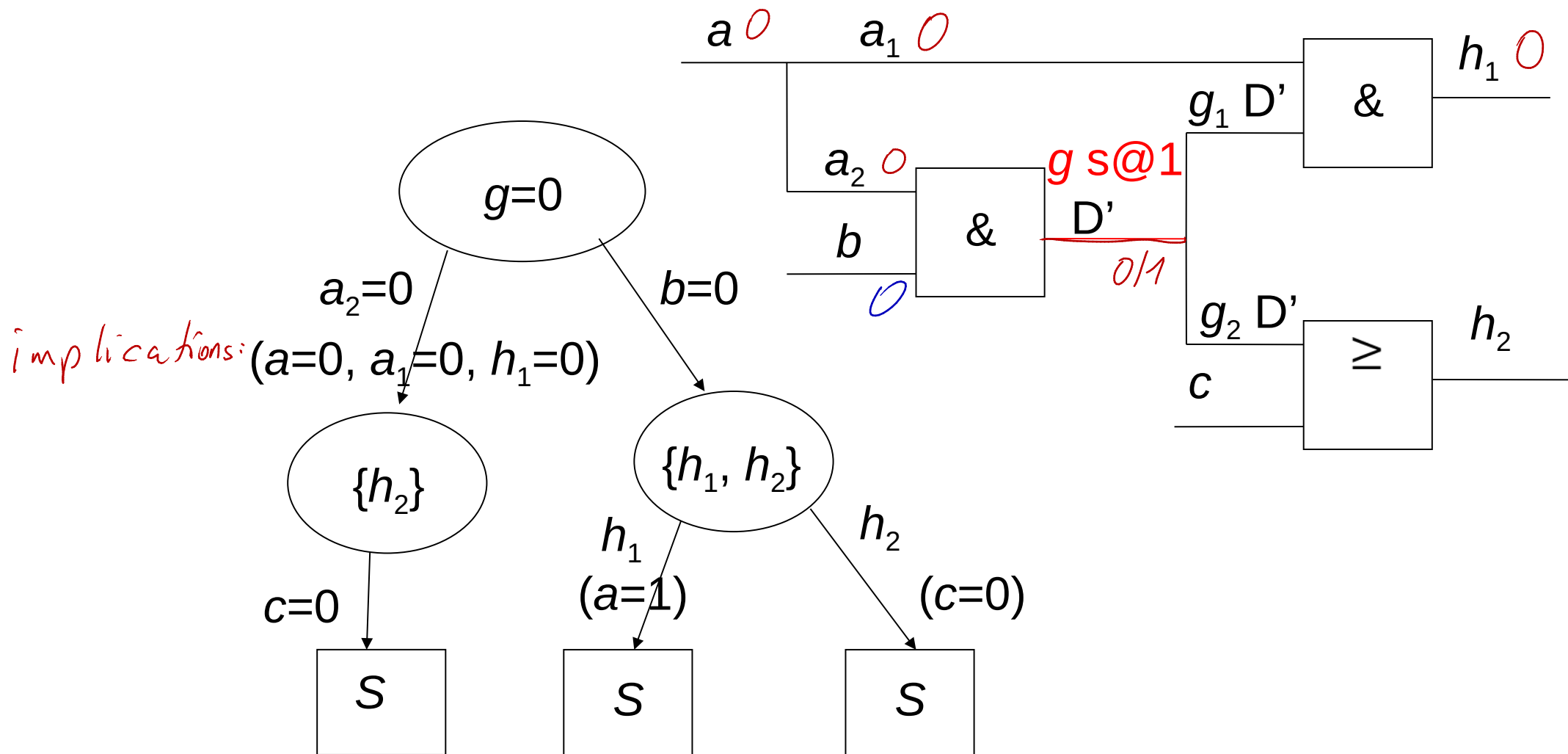


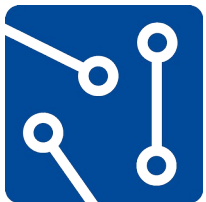
Decision Tree

- Used for implementing automatic test generation.
- Nodes represent sub-problems to be solved. In addition, SUCCESS and FAIL.
- **Branches** represent decisions.
- The ATPG algorithm traces the tree until all the possible paths end in FAIL, or until the first SUCCESS is encountered and there are no additional problems to solve.
- To find **all** the tests for a fault, find **all** the **SUCCESS** nodes.



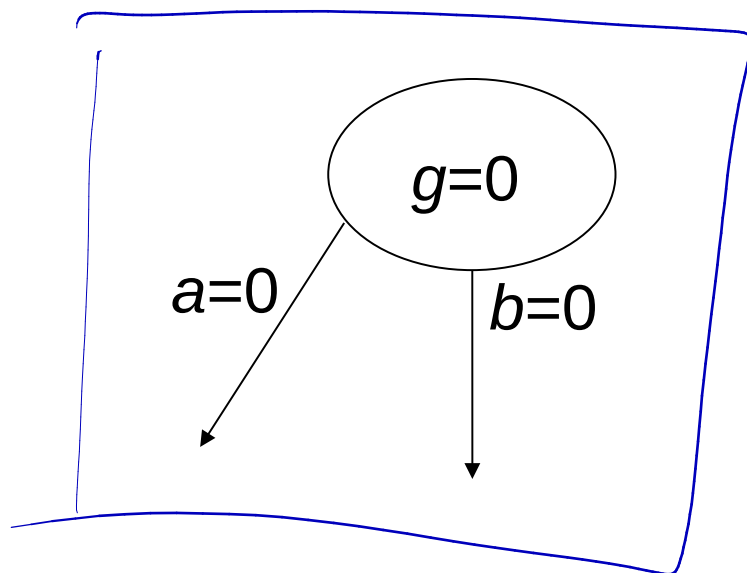
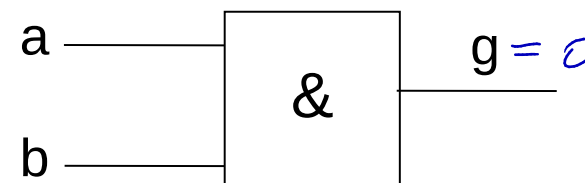
Example of a Decision Tree



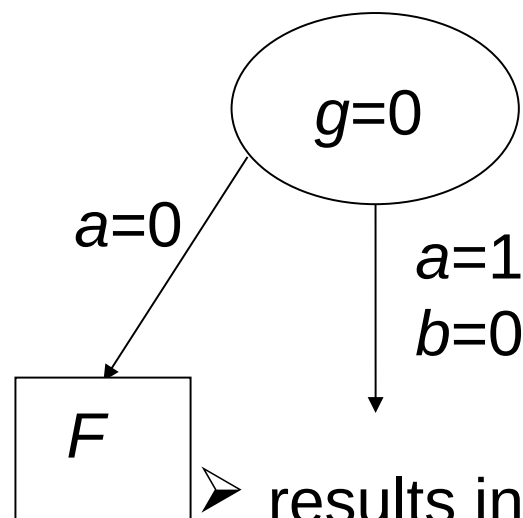


Reversing Incorrect Decisions

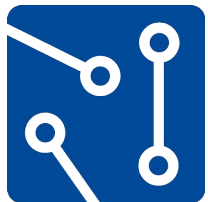
AND gate with inputs a , b and output g that needs to be set to 0.



If setting $a=0$ leads to a conflict:

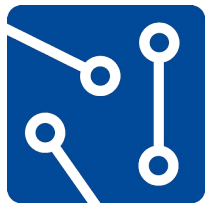


➤ results in more implications



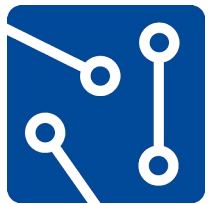
Complexity

- In the **worst case**, the size of the decision tree is **exponential** in the number of circuit lines.
- The worst case occurs if a **decision** has to be made **for every line**, and every decision has to be undone, with a different choice selected instead.



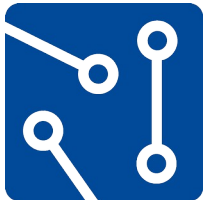
The D-Frontier

- One of the data structures maintained by ATPG procedures.
- The D-frontier contains all the gates whose **output value is x** but have **at least one D or D' on their inputs**.
- To propagate the fault, it is necessary to select one gate from the D-frontier and assign non-controlling values to the unspecified gate inputs.
- If the D-frontier is empty, it is necessary to backtrack. If no choices are left, the fault is undetectable.



The J-Frontier

- The J-frontier contains every gate whose **output value is known** but is **not implied by its input values** and does not imply its input values.
- An AND gate is entered into the J-frontier when its output needs to be set to 0, at least two of its inputs are x, and none of its inputs is 0.



The Implication Process `Imply()`

- Computes all the values that can be uniquely determined by implication.
- Maintains the D-frontier and the J-frontier.
- Uses the assignment queue: contains all the values that need to be implied. Its entries are of the form

$(l, v', \text{direction})$

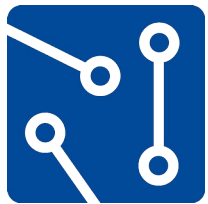
where direction is from {forward, backward}.

towards
outp.

towards
inputs

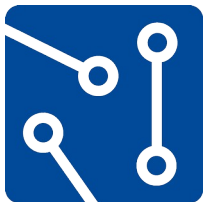
l $s-a-1$ 1

two entries:
 $(l, 0, \text{backw.})$
 $(l, 1, \text{forward})$

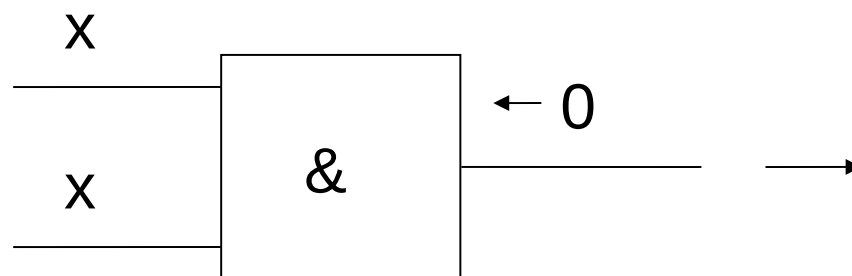
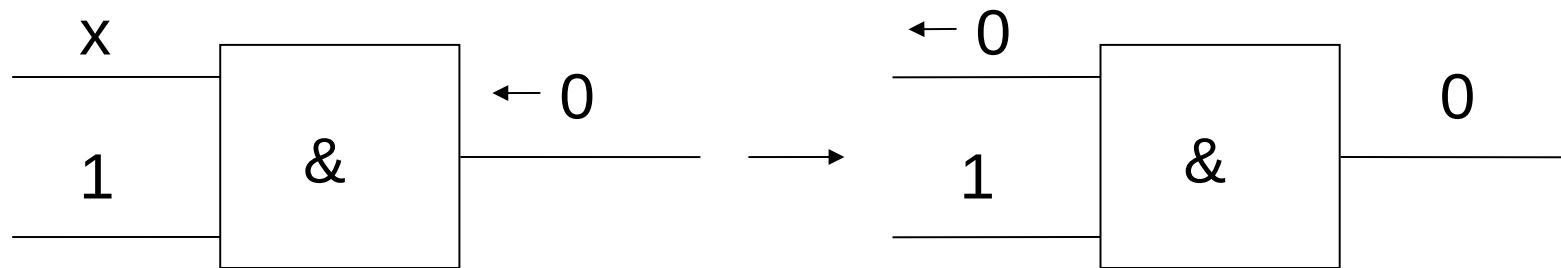


Initial Entries

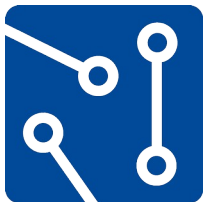
- For the fault l s.a.1, the **initial entries** in the assignment queue are $(l, 0, \text{backward})$ and $(l, D', \text{forward})$.
For l s.a.0, they are $(l, 1, \text{backward})$ and $(l, D, \text{forward})$.
- The implication procedure considers every entry in the **assignment queue**.
- When $(i, w', \text{direction})$ is considered:
 - If $i = w$, an inconsistency is detected and backtracking is initiated.
 - Exception: the faulty line with $(l, 0, \text{backward})$ and $(l, D', \text{forward})$, or $(l, 1, \text{backward})$ and $(l, D, \text{forward})$.



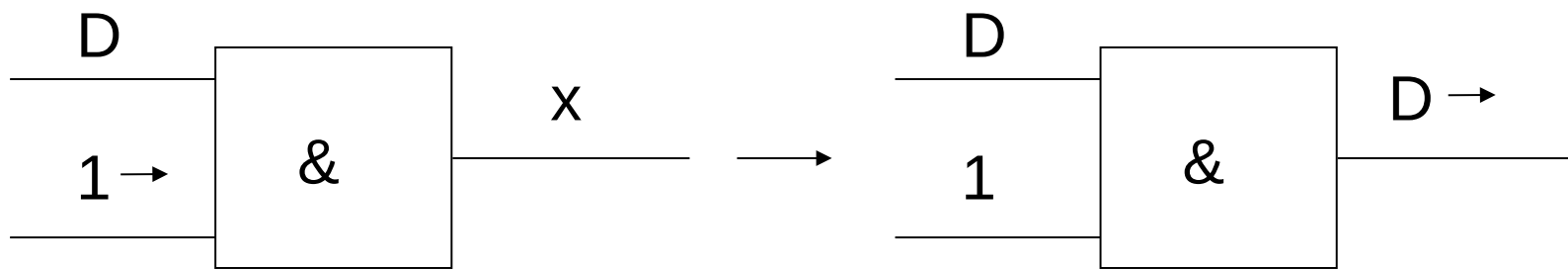
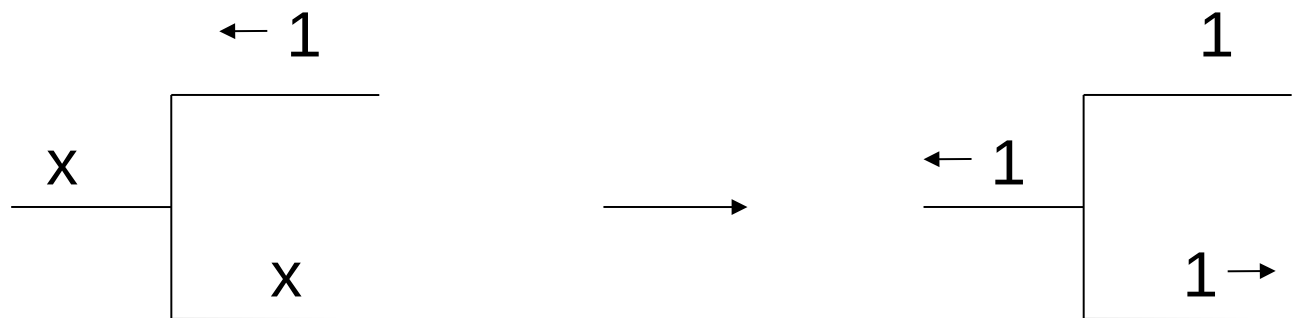
Examples of Implications



the gate
output is
added to the
J-frontier

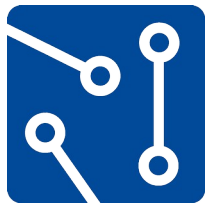


More Examples



gate is in the D-frontier

gate is removed from
the D-frontier

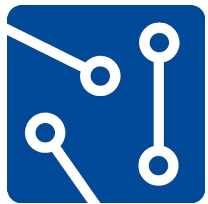


The D-Algorithm

■ D-Alg()

- if `imply()` = FAIL return FAIL
- if (error is not propagated to a PO)
 - ▶ if D-frontier is empty return FAIL
 - ▶ repeat
 - select an untried gate G from the D-frontier
 - c = controlling value of G
 - assign $i = c'$ (i.e., add to the assignment list (i, c', backward)) for every input i of G with value x
 - if D-Alg() = SUCCESS return SUCCESS
 - ▶ until all gates from D-frontier have been tried
 - ▶ return FAIL

Back-
tracking
occurs
here

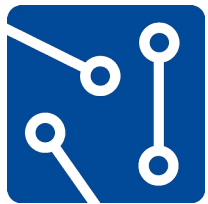


The D-Algorithm (cont'd)

■ /* error propagated to a PO */

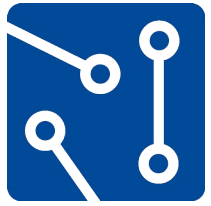
- if J-frontier is empty return SUCCESS
- select a gate G from the J-frontier
- c = controlling value of G
- repeat
 - ▶ select an input j of G with value x
 - ▶ assign j = c
 - ▶ if D-Alg() = SUCCESS return SUCCESS
 - ▶ assign j = c'
- until all the inputs of G are specified
- return FAIL

Back-
tracking
occurs
here



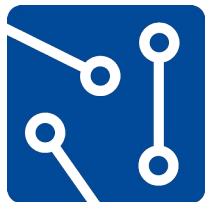
Reducing the Computation Time

- Implications can be used to avoid incorrect decisions.
- Computation time is **bounded** by imposing an upper bound on the **number of backtracks**
 - Limits the number of incorrect decisions that have to be remade
 - May lead to **incompleteness**!



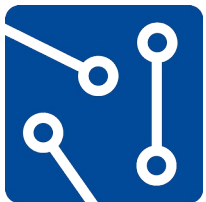
Additional Speed-up Techniques

- Global Implications
- X-Paths

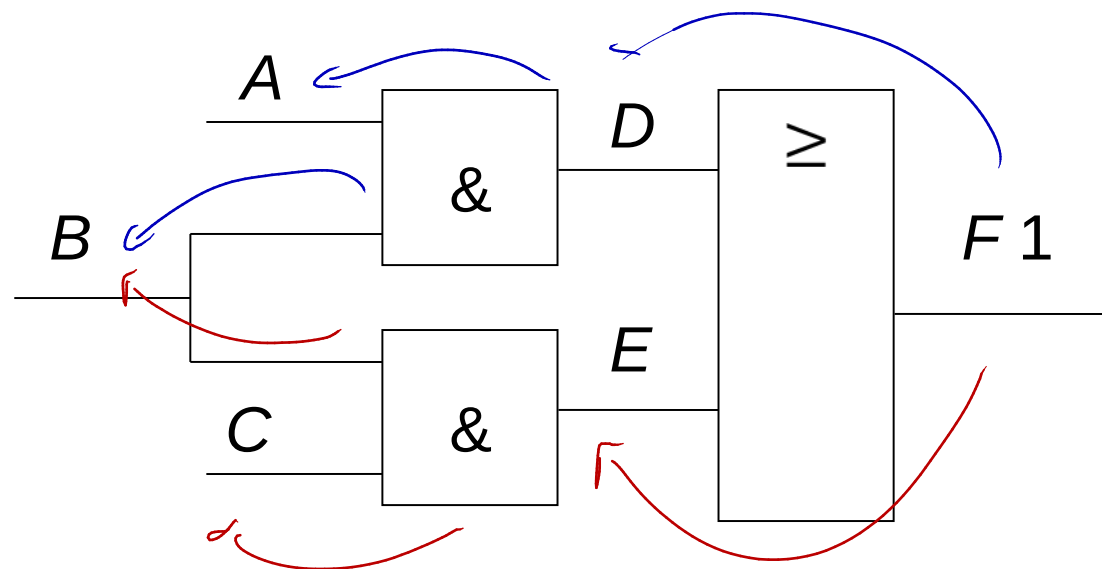


Local vs. Global Implications

- **Local implications** involve a line and its immediate successors or predecessors.
- **Global implications** involve a larger area of the circuit.
 - **Static learning** derives implications before any values are assigned.
 - **Dynamic learning** derives implications after some values have been assigned.

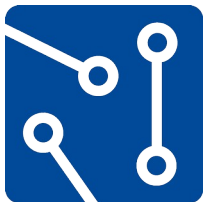


Example of Global Implications



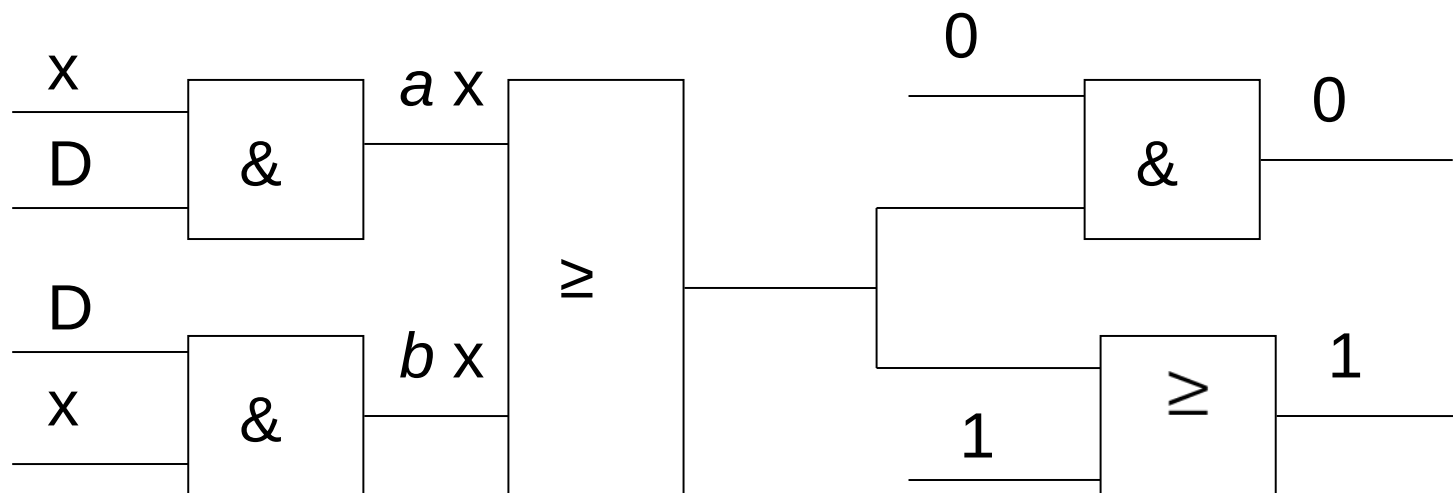
$F = 1$ implies $D = 1$ or $E = 1$. Both imply $B = 1$.
Therefore, $F = 1$ implies $B = 1$.

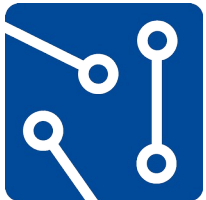
We cannot deduce this by local implications,
since $F = 1$ requires $D = 1$ or $E = 1$.



Error Propagation Look-Ahead

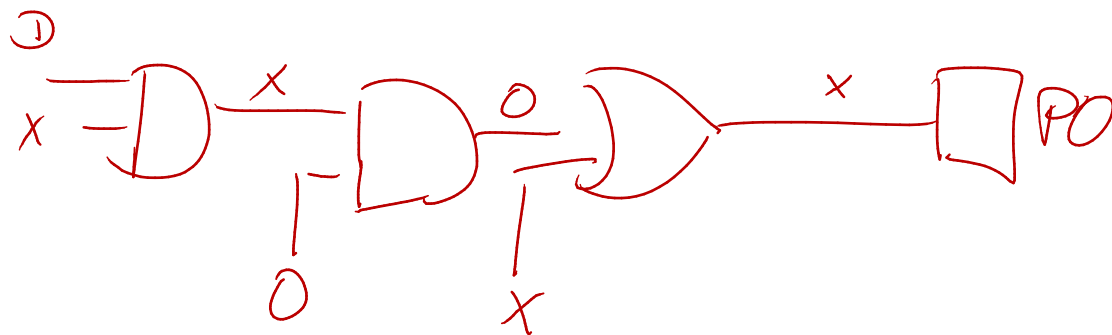
The D-frontier $\{a, b\}$ will eventually become empty

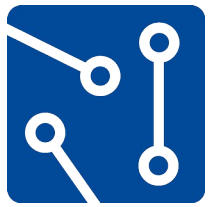




X-Paths

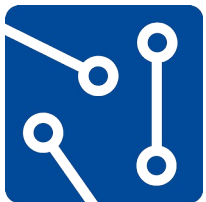
- **X-Path:** a **path** whose lines all have the **value x**.
- The fault effect from a gate in the D-frontier can propagate to a primary output only if there is an x-path from the gate output to a primary output.
- If no x-path exists for any gate in the D-frontier, backtracking can be done without making any additional decisions.



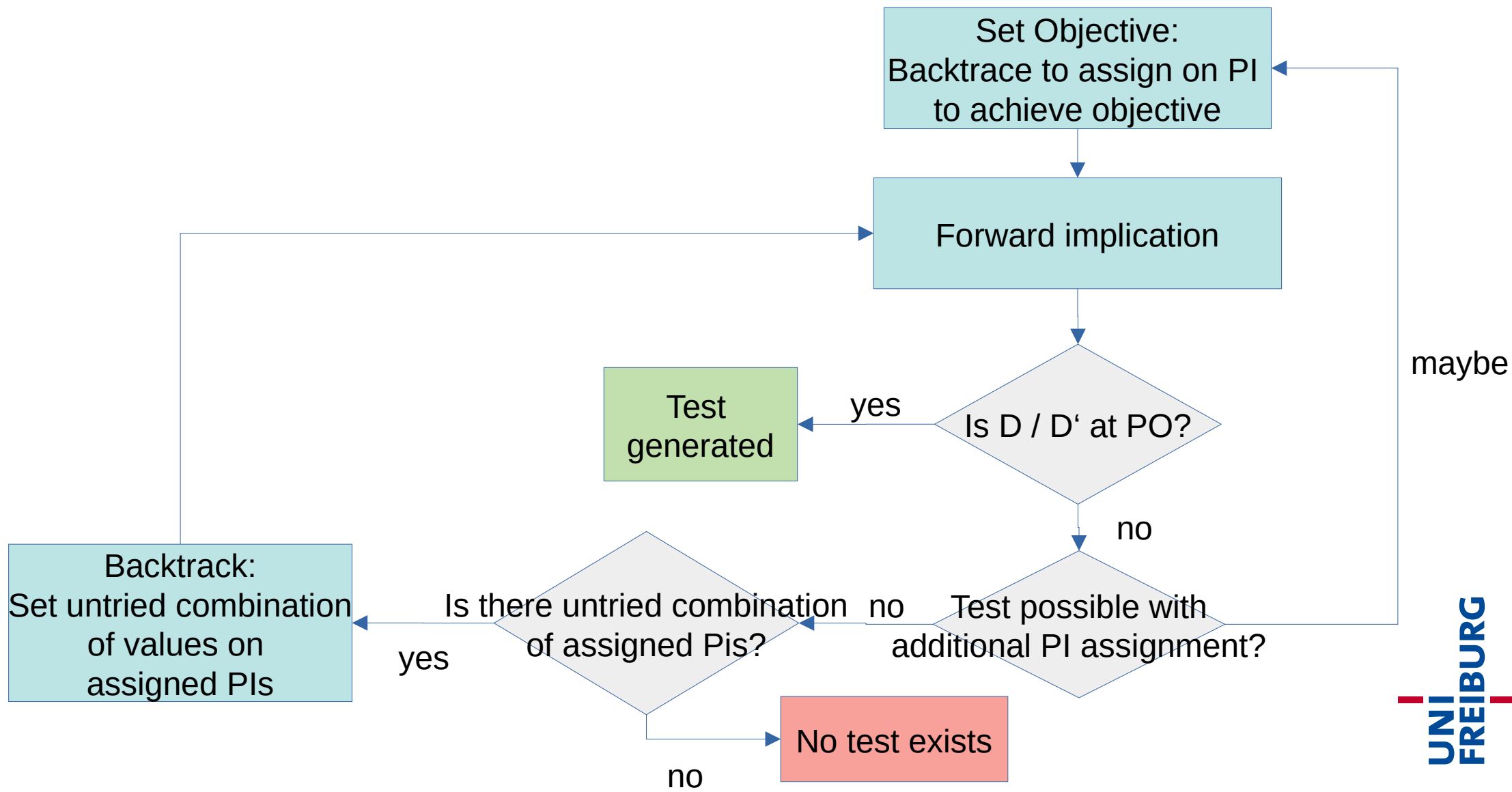


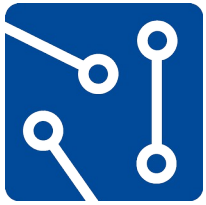
PODEM (Path-Oriented Decision Making, Goel 1981)

- Alternative algorithm for ATPG
- **Main Ideas:**
 - Allow decisions to assign primary inputs only
 - Don't assign internal lines
 - Greatly reduces size of search space
 - Assigned PIs are then propagated forward only
 - No justification needed (why?)
 - Flip last decision (PI assignment) on two conditions:
 - Fault not activated
 - Fault not propagated to any output



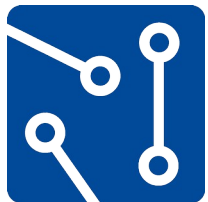
PODEM Flowchart





Components of PODEM

- Determine an objective
 - If D / D' has not appeared at the fault site,
 - set objective to activate fault
 - If D / D' has appeared,
 - set objective to propagate fault to primary output
- Given objective, determine PI value
 - Backtrace to PI
 - Backtrack if conflict occurs
- **Backtrace is different from backtrack!**
 - **Backtrace** goes back to a PI from a certain signal in netlist
 - **Backtrack** goes back to last decision and flips it



Procedure ***Backtrace***(k, w)

*objective : assign line k to
value w*

$v = w$

while k is a gate output:

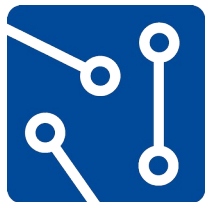
i = inversion of k

select an input j of k with value x

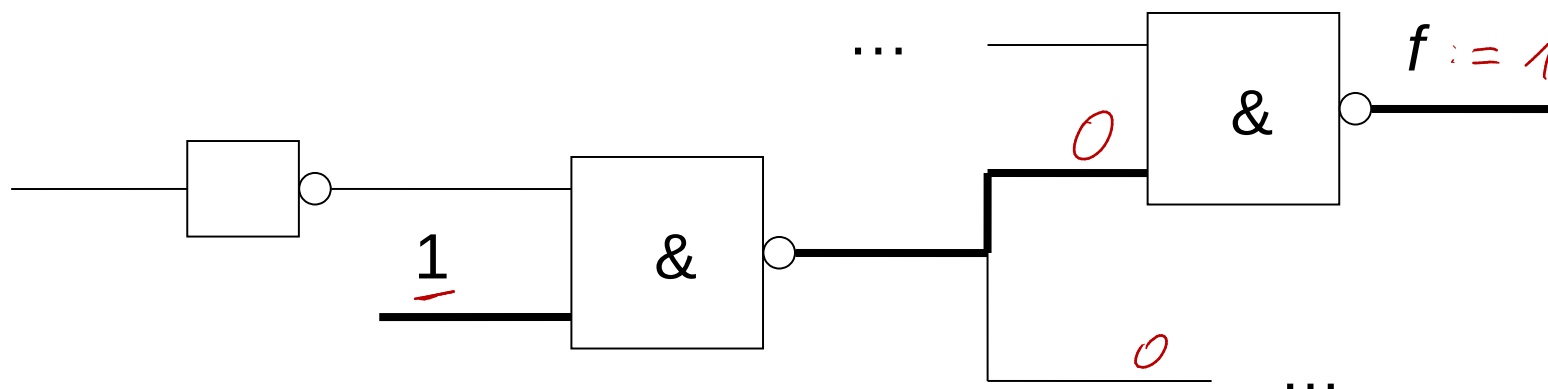
$v = v \oplus i$

$k = j$

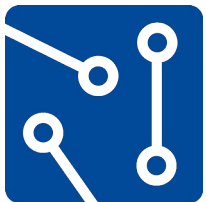
/* k is a PI */ return (k, v)



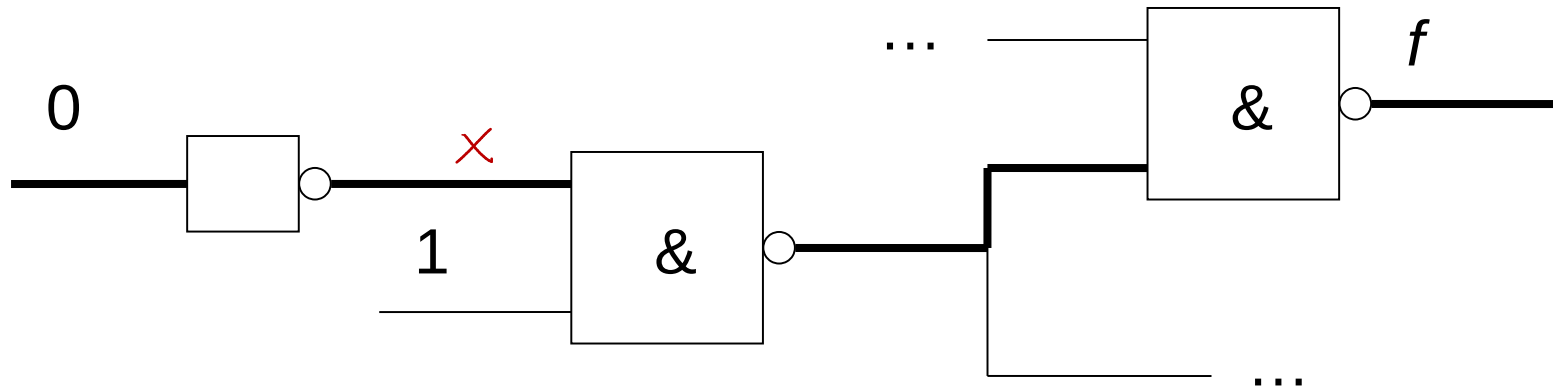
Example: *Backtrace*(f , 1)



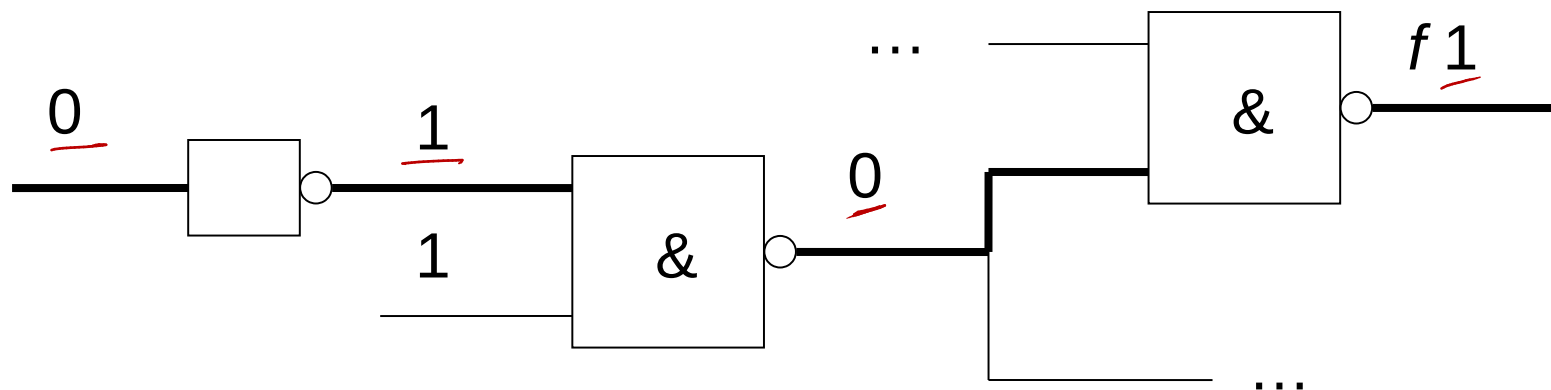
no values are implied

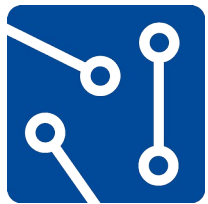


Backtrace(f, 1) again



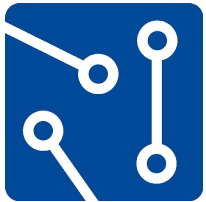
implications:





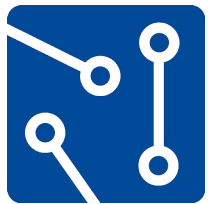
Selecting Objectives

- An **objective** – a pair (i, v) such that line i needs to be set to the value v .
- For the fault l s.a. v :
 - if the value of l is x , return (l, v') .
 - select a gate G from the D-frontier.
 - select an input j of G with value x .
 - c = controlling value of G .
 - return (j, c') .
- Is there a need for justification objectives? ||



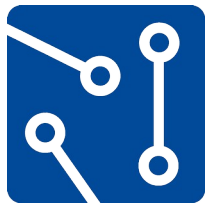
Overall Structure of PODEM

- *if* error at PO return **SUCCESS**
- *if* test not possible return **FAIL**
- $(k, v) = \text{Objective}()$
- $(j, w) = \text{Backtrace}(k, v)$
- $\text{Imply}(j, w)$
- *if* **PODEM()** = **SUCCESS** return **SUCCESS**.
- */* backtracking */* $\text{Imply}(j, w')$
- *if* **PODEM()** = **SUCCESS** return **SUCCESS**.
- */* cancel the decision on j and backtrack */* $\text{Imply}(j, x)$
- return **FAIL**



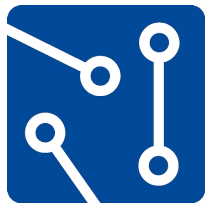
Unique Features of PODEM

- Nodes in the decision tree correspond only to **primary inputs**.
- Only forward implications of primary input values are needed, and no backward implications are done.
- There is **no need to store a J-frontier** since a single objective is selected based on the D-frontier and immediately justified.
 - ~ No conflicts occur due to forward implications.



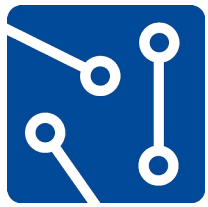
Backtracking in PODEM

- Requires that a primary input value would be changed. The new value is implied to compute the new circuit values.
- In the D-algorithm, it was necessary to store the values of **all the lines** at every decision point in order to be able to undo the decision.

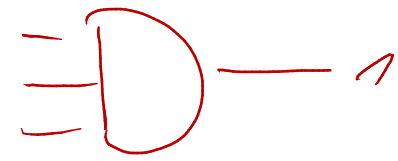
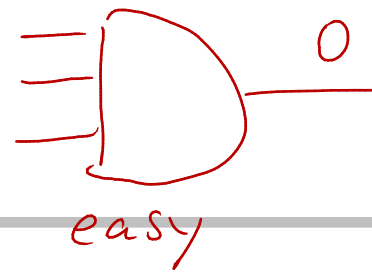


Selection Criteria in ATPG Procedures

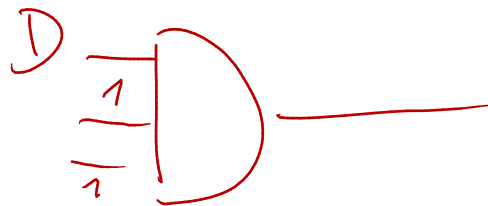
- Among different unsolved problems ($l_1 = v_1, l_2 = v_2, \dots$), which all need to be solved, **select the most difficult one**.
 - Avoids wasting time on easier problems if a difficult one cannot be solved.
- Among different choices for the same problem, **try the easiest one first**.
 - Avoid a long search if a shorter one can solve the problem.
- **What does that mean for the D-Algorithm and what for PODEM?**

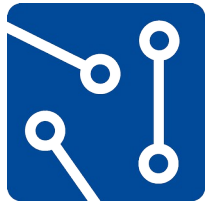


Measuring Difficulty



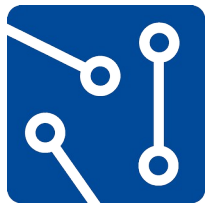
- **Controllability** measures
 - Indicate the **difficulty of setting a line** to a given value.
 - Used to select the most difficult line-justification problem, and to select among unspecified inputs of a gate the one that is easiest to set to the controlling value.
- **Observability** measures
 - Indicate the **difficulty of propagating an error** from a line to a PO.
 - Used to select a gate from the D-frontier whose error is easiest to propagate.



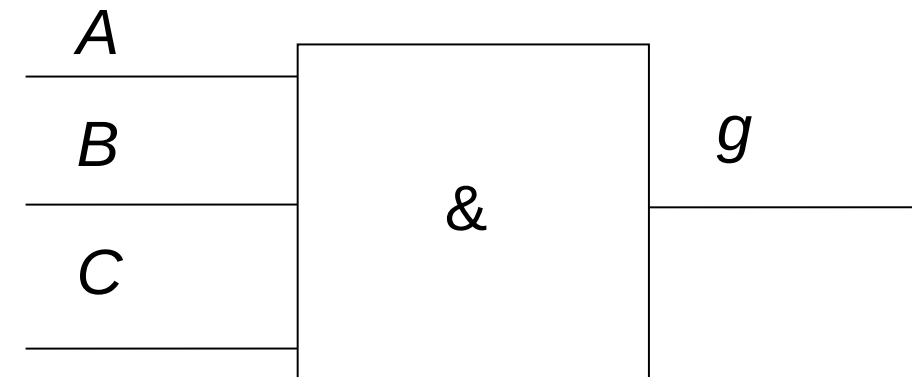


Example of C/O Measures

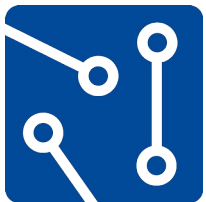
- For **every line** l :
 - $C0(l)$ is the difficulty of setting $l = 0$.
 - $C1(l)$ is the difficulty of setting $l = 1$.
- For a **primary input**, $C0(l) = C1(l) = 1$ (the lowest value).
- $C0(l)$ and $C1(l)$ for other lines are computed by traversing the circuit from inputs to outputs.



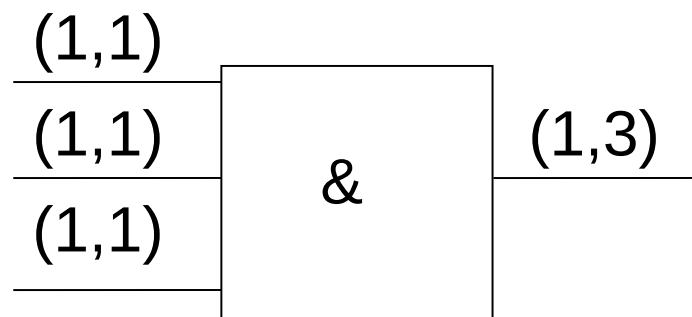
AND Gate



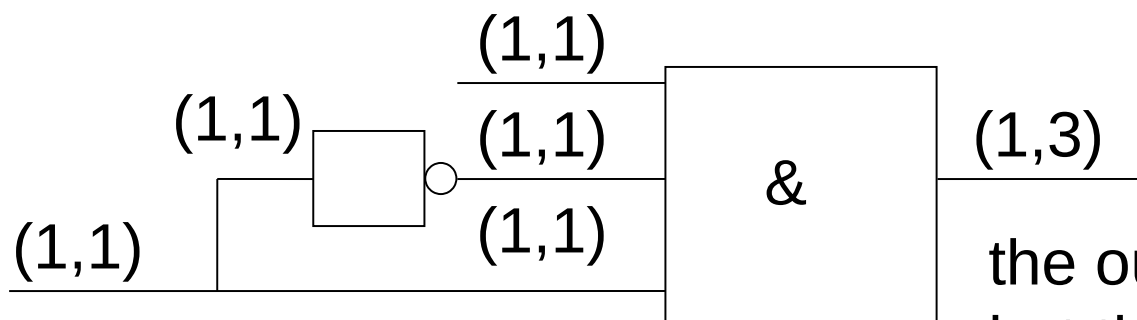
- $C0(g) = \min\{C0(A), C0(B), C0(C)\}$
- $C1(g) = C1(A) + C1(B) + C1(C)$
- **Inaccurate** when the inputs of a gate are driven by the same lines.
 - Tolerated to simplify the computation
 - Reminder: The purpose is just to speed up the D-algorithm!



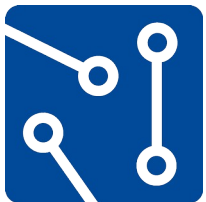
Non-Independent Inputs



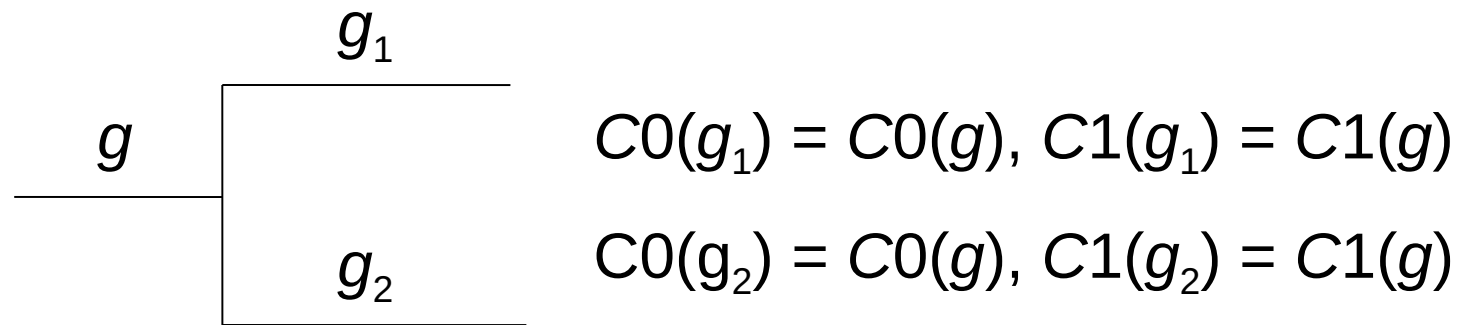
Notation:
(C0(I), C1(I))



the output is always 0,
but the measures do not
reflect that it is
impossible to set it to 1.

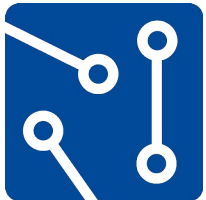


Fan-out Branches



Inverters



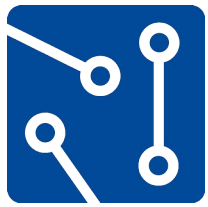


Observability Measures

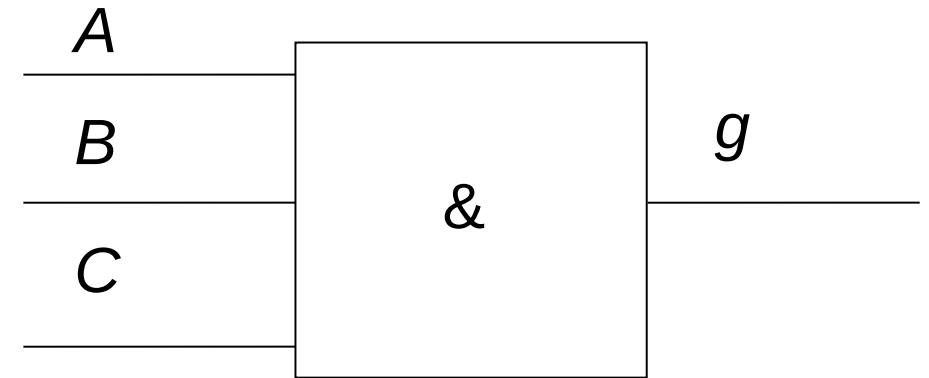
- For **every line l**:
 - $O(l)$ is the difficulty of propagating an error from l to a PO.
- For a **primary output**, $O(l) = 0$ (the lowest value).
- $O(l)$ for other lines is computed by traversing the circuit from outputs to inputs, using the controllability measures.



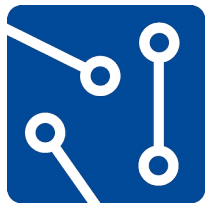
$$O(a) = c1(b) + c1(c) + o(d)$$



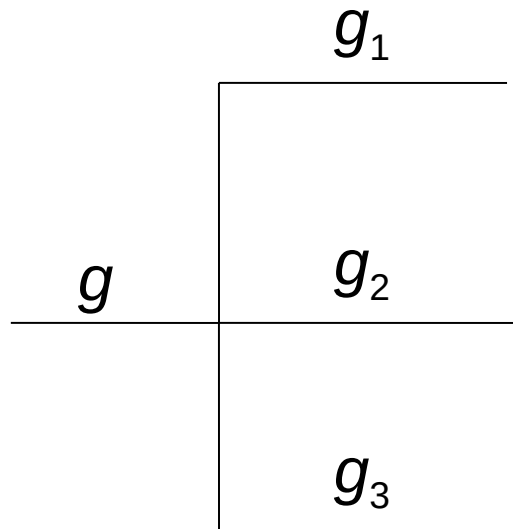
AND Gate



- $O(A) = C1(B) + C1(C) + O(g)$
- Inaccurate when the inputs of a gate are driven by the same lines
 - Tolerated to simplify the computations

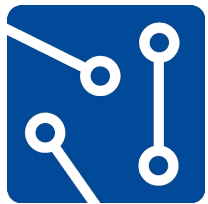


Fan-out Stems



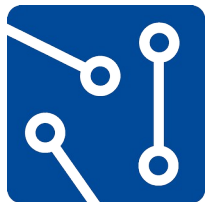
$$O(g) = \min\{O(g_1), O(g_2), O(g_3)\}$$

Based on the assumption that we will select the easiest path to propagate fault effects from g to a primary output.



Overall Structure of an ATPG Algorithm

- Set of target faults F
- Use one or more of the following phases:
 - **Phase 1:** Low cost, fault-independent test generation. Typically, random patterns. Drop a subset of faults F_{rand} from F .
 - **Phase 2:** Fast identification of undetectable faults. Drop a subset of faults F_{undet} from F .
 - **Phase 3:** Medium cost, (fault-oriented) simulation-based test generation.
 - **Phase 4:** Deterministic, fault-oriented test generation (use an increasing backtrack limit).
 - **Phase 5+:** Test set optimization

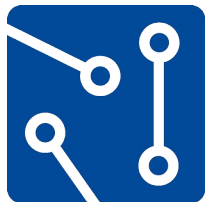


Unspecified Values

- A test t for a target fault f may be **incompletely specified**.
- **Optimization** possible by specifying the unspecified values
 - May increase the number of **faults detected** by t (accidental detections)
 - May increase test set **compactness**
 - May reduce **energy consumption**

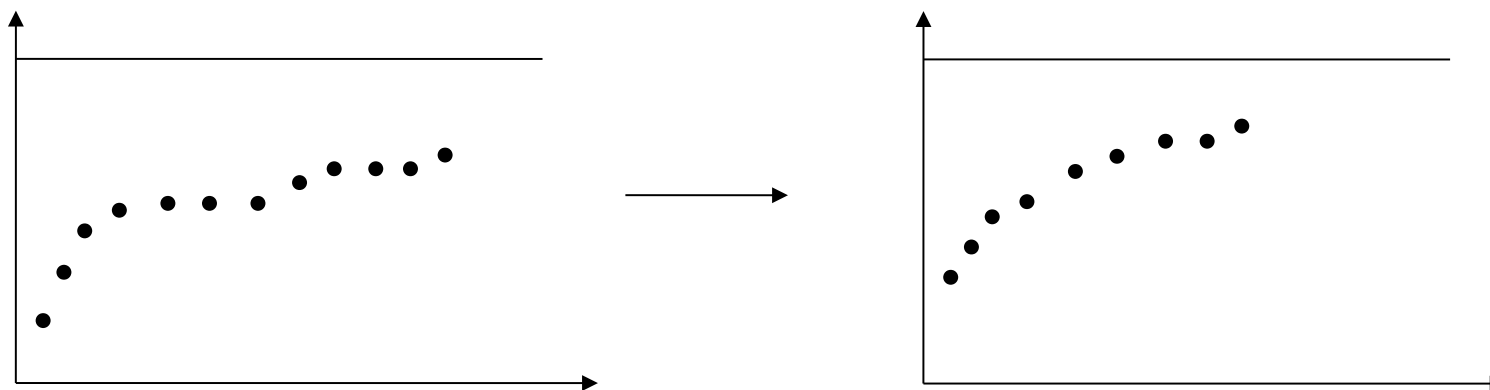
Fault 1: 0 x 1 1 x x x
Fault 2: 0 1 x 1 0 x x

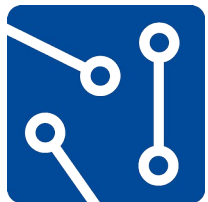
0 1 1 1 0 x x



Test Compaction

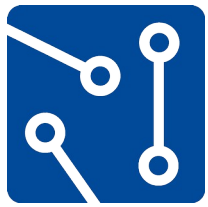
- Reduces the number of tests required to achieve the target level of fault coverage.
 - Reduces cost of test application
 - ↘ Less test application time
 - ↘ Less tester memory requirements





Classes of Test Compaction Procedures

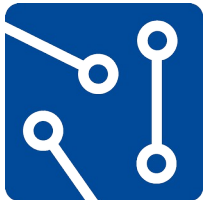
- **Static compaction:**
 - Performed **after** test generation is complete
 - └ Reduces the size of an **existing** test set
 - Does not require any modifications to the test generation procedure
 - Applicable following any test generation procedure
 - Can typically be done **efficiently**



Classes of Test Compaction Procedures

- **Dynamic compaction:**

- Test set size is considered **during** the test generation process.
- The test generation process is modified to produce small numbers of tests
- Typically, tries to maximize the number of faults detected by every test
 - ↗ Minimizes the total number of tests required to detect all the faults
- More **effective** than static compaction



Static Compaction Based on x-Values

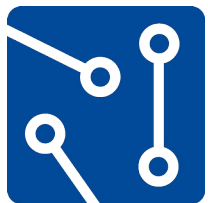
- This technique is applicable to test generation procedures that produce incompletely specified tests (no filling of x values).
- Two tests are **compatible** if they do not specify opposite values for any PI.

$\begin{pmatrix} 0 & x & 1 & x & x & 0 \\ 0 & 1 & 1 & x & 1 & 0 \end{pmatrix}$ ✓

- Examples: 00xx and 0x1x are compatible.
- 0xxx and 1xxx are incompatible.
- Two compatible tests t_i and t_j can be combined into a single test $t_i \sqcup t_j$.

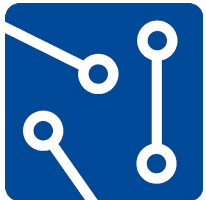
- Example: 00xx and 0x1x can be combined into 001x.

$\begin{pmatrix} 0 & x & 1 & x & x & 0 \\ 1 & x & x & 1 & 1 & 0 \end{pmatrix}$



Intersection

$\models$	0	1	x
0	0	-	<u>0</u>
1	- <i>not compat.</i>	1	1
x	0	1	x



Optimal Procedure

■ $t_1 = 01x$

■ $t_2 = 0x1$

■ $t_3 = 0x0$

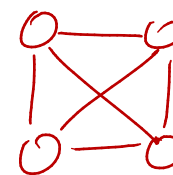
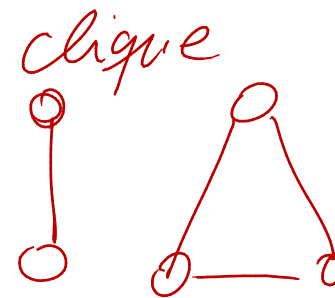
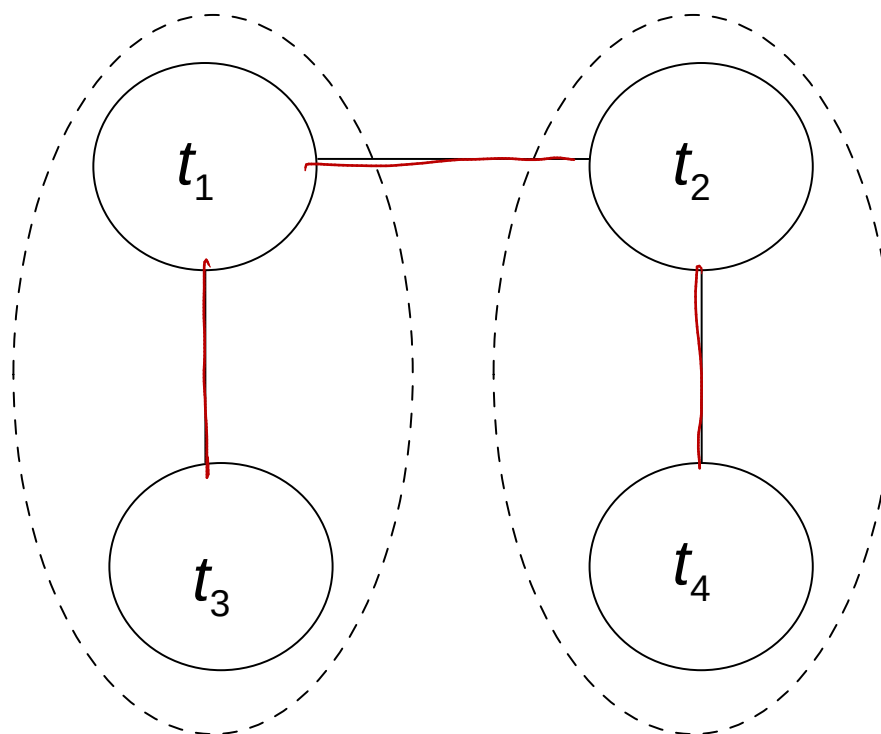
■ $t_4 = x01$

■ Use:

- $t_1 \sqsupseteq t_3 = 010$

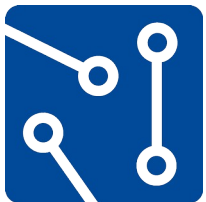
- $t_2 \sqsupseteq t_4 = 001$

Compatibility graph:



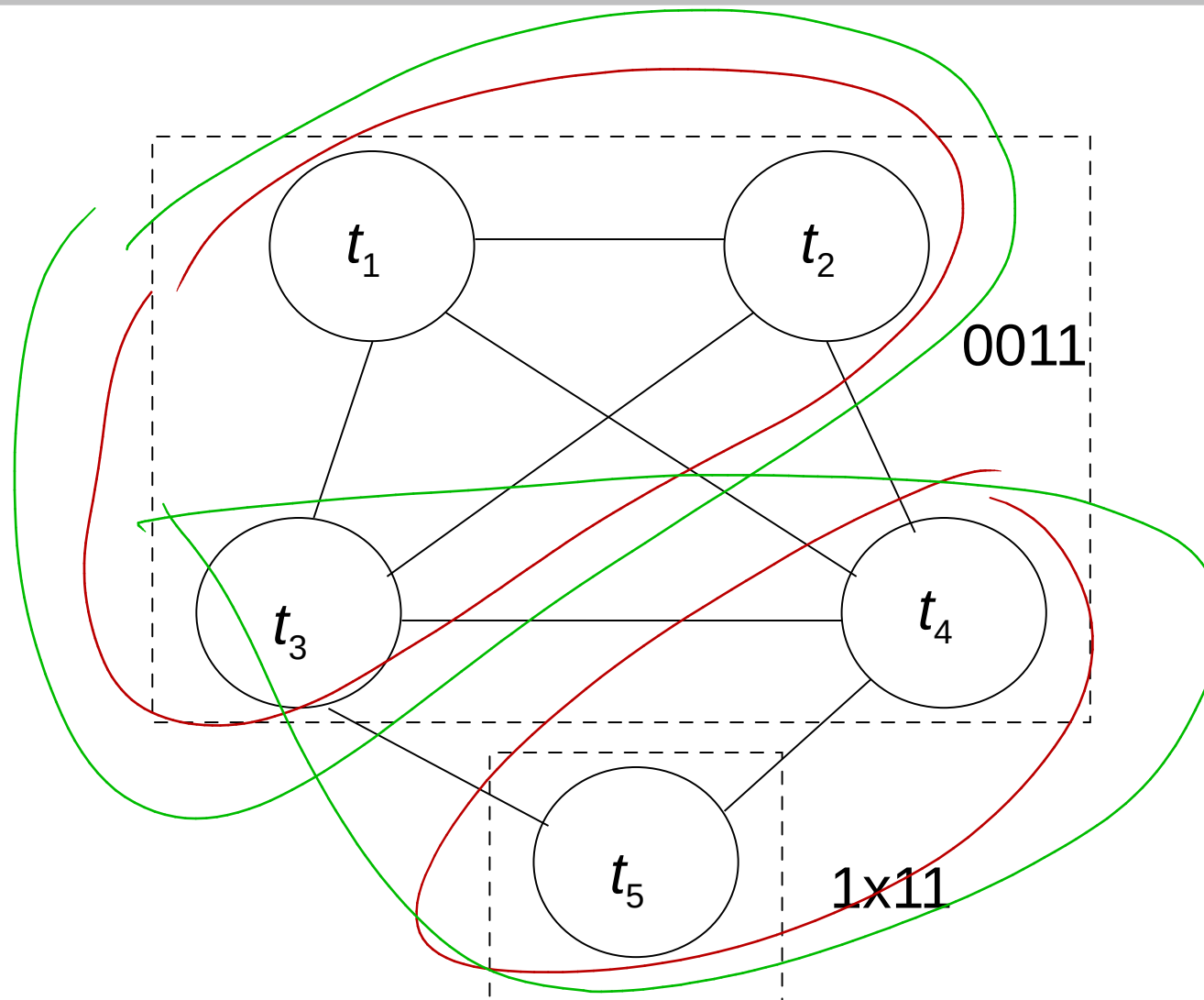
each clique is
a set of pairwise
compatible patterns
→ can be merged
into a single
pattern

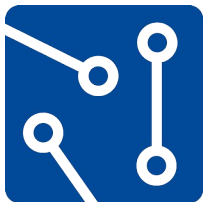
→ cover the nodes of the graph with a minimum
number of cliques



Another Example of the Optimal Procedure

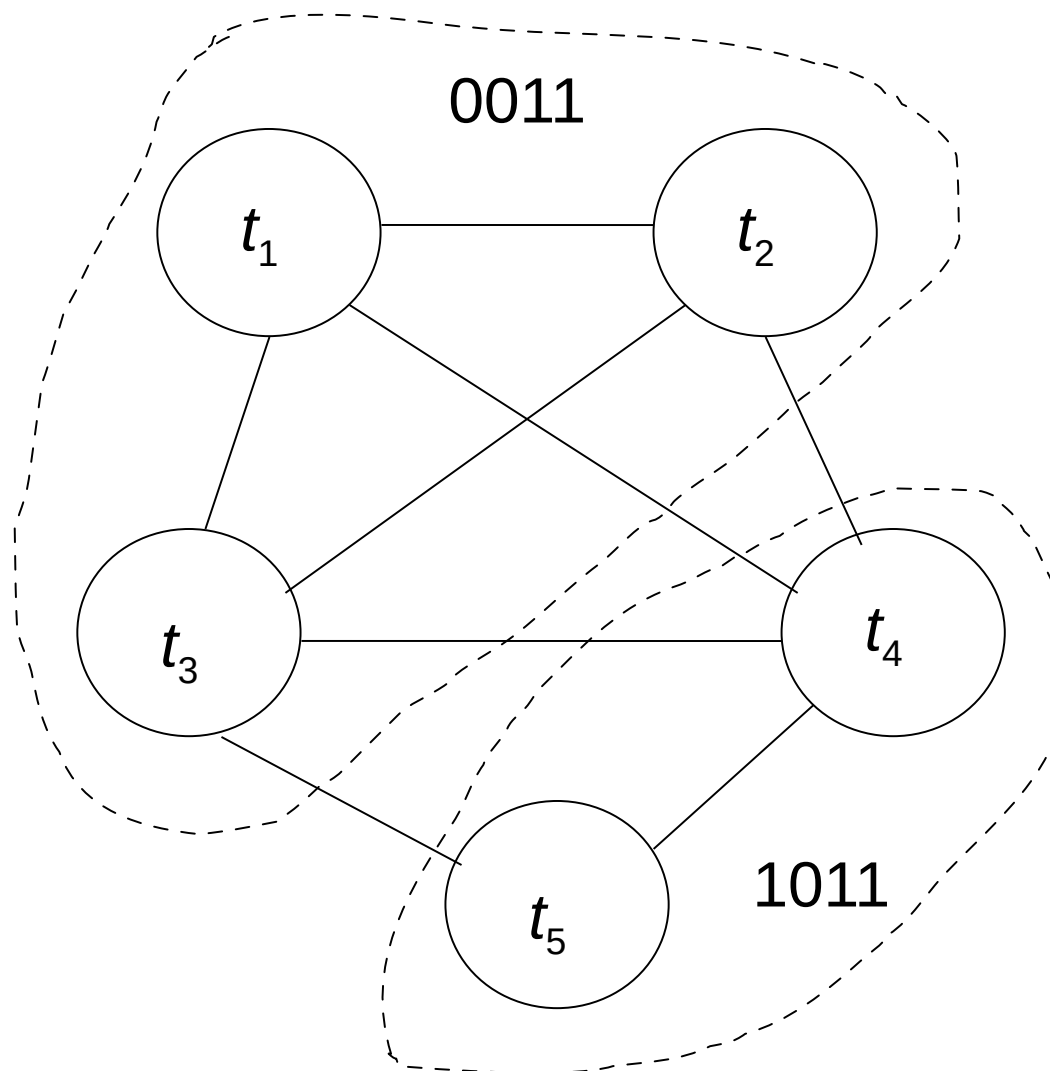
- $t_1 = 00xx$
- $t_2 = 0x1x$
- $t_3 = x0x1$
- $t_4 = x011$
- $t_5 = 1x11$

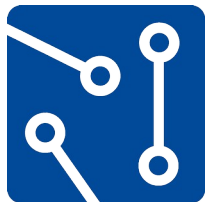




Another Solution

- $t_1 = 00xx$
- $t_2 = 0x1x$
- $t_3 = x0x1$
- $t_4 = x011$
- $t_5 = 1x11$

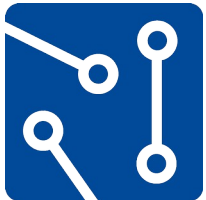




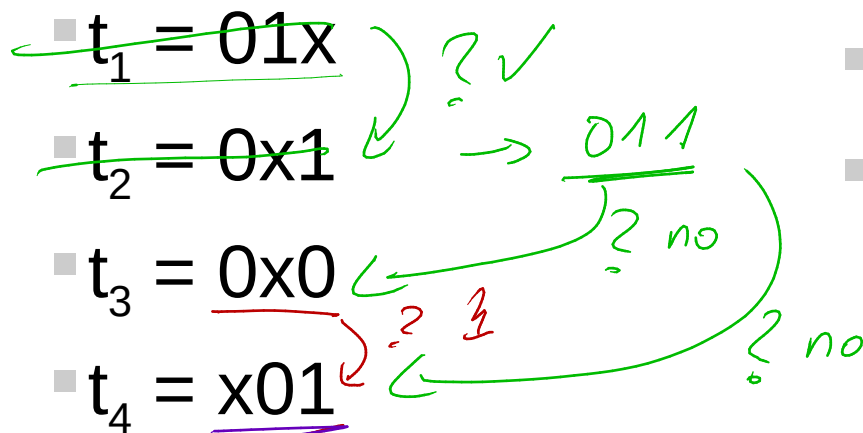
General Optimal Procedure

- A **clique** (complete subgraph) contains all the edges between its vertices
- All the tests corresponding to the vertices of a clique can be combined
- Find the **compatibility graph**
- Cover the graph using a minimum number of cliques
Problem is NP-complete → apply efficient heuristics

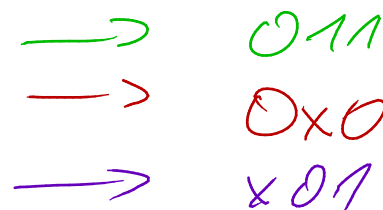
*not necessarily
disjoint*



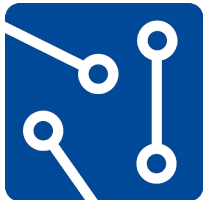
Suboptimal Solution (Greedy Procedure)



- Combine t_1 and t_2 into 011
- No other tests can be combined



- Advantage over optimal solution: **run time**
 - no need to consider all the test pairs
 - no need to find maximum compatible subsets



Another Example of the Greedy Procedure

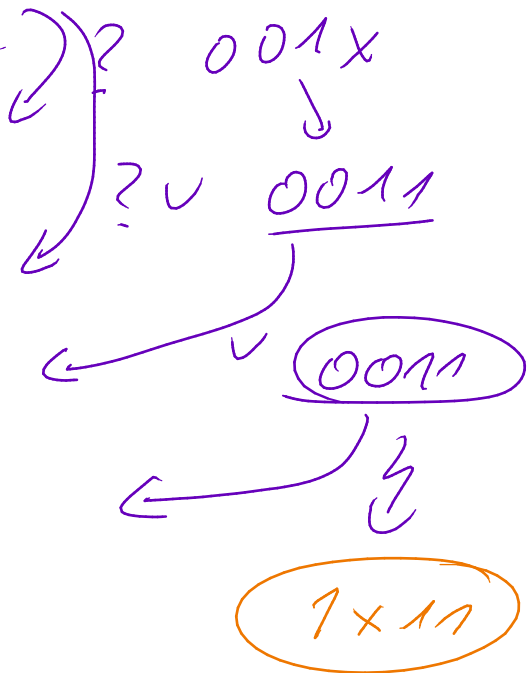
■ $t_1 = \underline{00}xx$

■ $t_2 = 0x\underline{1}x$

■ $t_3 = x0x\underline{1}$

■ $t_4 = x0\underline{11}$

■ $t_5 = \underline{1x11}$

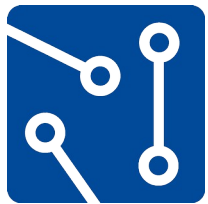


¬ Combine t_1 and t_2 into $001x$.

¬ Combine $001x$ and t_3 into 0011 .

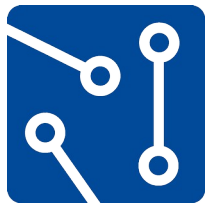
¬ Combine 0011 and t_4 into 0011 .

¬ Result: $0011, 1x11$.



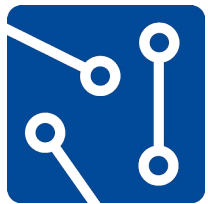
Reverse Order Fault Simulation

- Applied to a (staticly compacted) test set $T = \{t_1, t_2, \dots, t_n\}$
- Suppose that the tests were generated in the order $\langle t_1, t_2, \dots, t_n \rangle$
- Each test t_i was fault simulated after it was generated. Let the set of faults detected by t_i be F_i
- It is possible that $t_{i+1}, t_{i+2}, \dots, t_n$ (generated after t_i) detect all the faults in F_i
- This can be identified by simulating the test set in reverse order $\langle t_n, t_{n-1}, \dots, t_1 \rangle$



The Procedure

- Let F be the set of target faults detected by $T = \{t_1, t_2, \dots, t_n\}$.
Set $i = n$.
- Simulate F under t_i and find the set of faults it detects, R_i
 - If R_i is empty, remove t_i from T
 - Else, remove R_i from F
- Set $i = i - 1$. If $i > 0$, go to Step 2

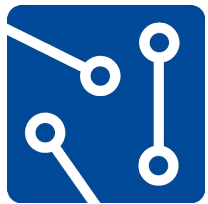


Example

- t_0 detects f_0
- t_1 detects f_1, f_2, f_3
- t_2 detects f_2, f_4
- t_3 detects f_2, f_3, f_5
- t_4 detects f_1, f_4, f_6
- t_5 detects f_5, f_6, f_7

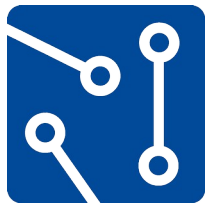
Reverse order simulation:

- t_5 detects f_5, f_6, f_7 ✓
- t_4 detects $f_1, f_4, \cancel{f_6}$ ✓
- t_3 detects $f_2, f_3, \cancel{f_5}$ ✓
- ~~t_2 detects $\cancel{f_2}, \cancel{f_4}$~~
- dropped
- t_1 detects $\cancel{f_1}, \cancel{f_2}, \cancel{f_3}$
- dropped
- t_0 detects f_0 ✓



Forward-Looking Reverse Order Fault Simulation

- During forward-order simulation with fault dropping, **keep** for every fault the first test that detects it.
- During the **reverse** order fault simulation process, if t_i is not the first test to detect any yet-undetected fault, drop t_i without simulating it.
- t_i is substituted by earlier tests that detect the yet-undetected faults detected by t_i .



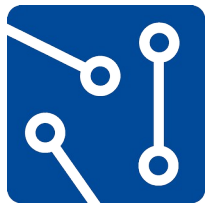
Example

fault	tests
f_1	$\underline{t_1}$
f_2	$\underline{t_2}, t_3$
f_3	$\underline{t_3}$
f_4	$\underline{t_1}, t_2$

test	faults
t_1	$\underline{f_1}, \underline{f_4}$
t_2	f_2 , (f_4)
$\underline{t_3}$	$f_2, \underline{f_3}$



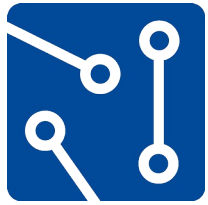
- t_3 is the first test that detects f_3 . Simulate t_3 . Keep it because of f_2, f_3 .
- t_2 it not the first test to detect any yet-undetected fault. Skip over it without simulation (f_4 has a later test that will detect it).
- t_1 is the first test that detects f_1, f_4 . Simulate t_1 . Keep it because of f_1, f_4 .



Results for Industrial Circuits

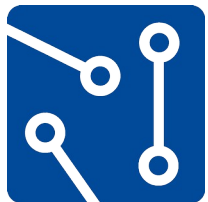
			reverse		forward-looking		
circuit	faults	orig tests	tests	CPU	tests	drop	CPU
ckt1	16641	422	358	5.02	339	80	4.54
ckt2	64719	1737	1303	21.08	1226	504	16.23
ckt3	58198	3719	3040	10.28	2797	877	10.16
ckt4	116553	3440	2780	38.03	2597	839	32.53
ckt5	118306	1257	1020	24.35	938	315	20.66
ckt6	1846313	28384	21858	3655.07	20258	8117	3593.87

[I. Pomeranz, S.M. Reddy, X. Lin: „Experimental Results of Forward-Looking Reverse Order Fault Simulation on Industrial Circuits with Scan“ Asian Test Symposium 2001]



Dynamic Compaction

- Secondary target faults:
 - After a test t is generated for a target fault f (primary target fault):
 - ↗ Select a yet-undetected fault f_1 and use only the unspecified values of t to detect f_1 .
 - ↗ Repeat for yet-undetected faults $f_2, f_3, \dots$
 - ↗ $f_1, f_2, f_3, \dots$ are called secondary target faults.
- During test generation for a **secondary target** fault, the initial specified values of t are fixed and no backtracking occurs on these values.



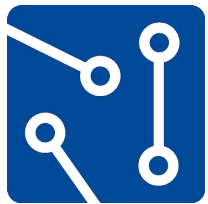
Example

Initially: $t = \text{xxxxxxx}$

After targeting f : $t = \underbrace{001}_{\text{fixed}}\text{xxxxx}$

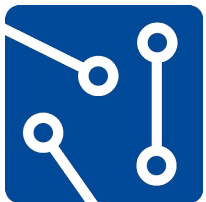
After targeting f_1 : $t = \underbrace{00101}_{\text{fixed}}\text{xxx}$

After targeting f_2 : $t = \underbrace{001010}_{\text{fixed}}\text{xx}$



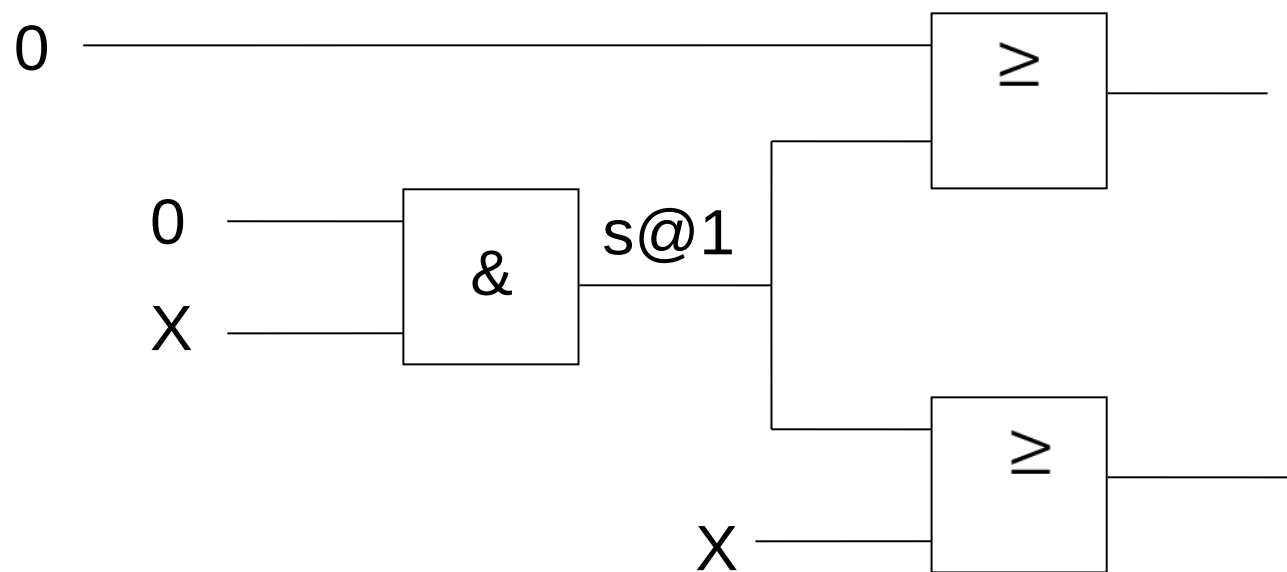
Decision Tree

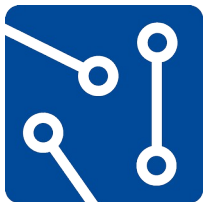
- $t = \text{xxxxxxxx}$: Decision tree can include any input.
- $t = 001\text{xxxxx}$: Decision tree does not include the first three inputs.
- $t = 00101\text{xxx}$: Decision tree does not include the first five inputs.
- $t = 001010\text{xx}$: Decision tree does not include the first six inputs.



Enhanced Dynamic Compaction

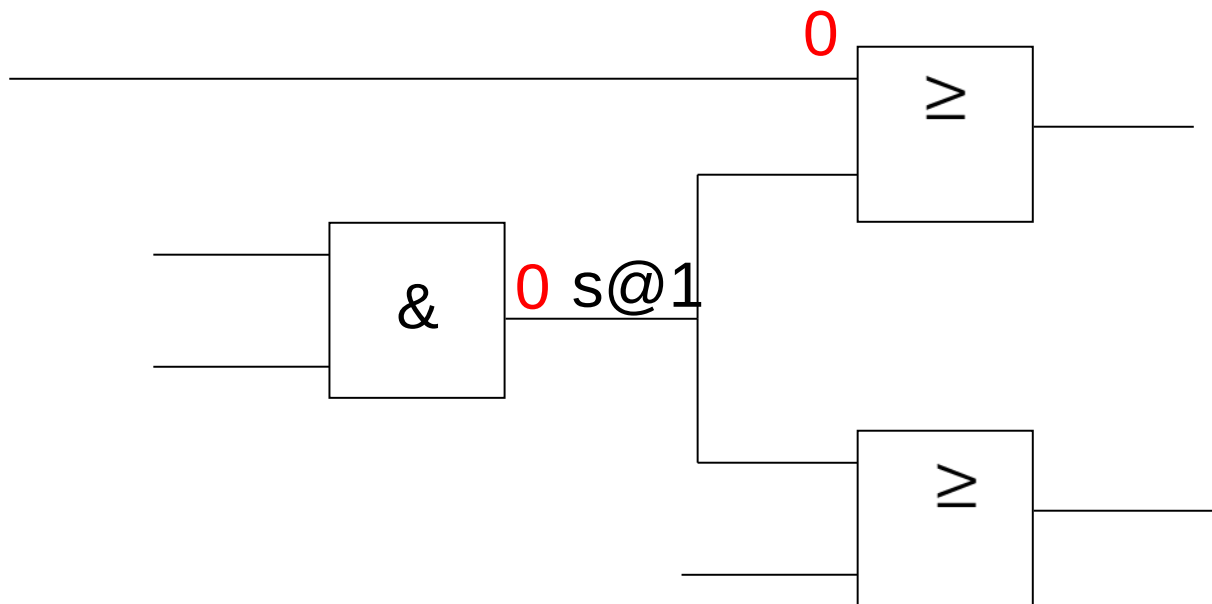
- Focus on internal values instead of inputs
 - 00XX is a test for the s@1 fault
 - Not compatible with tests 1100 or 0100

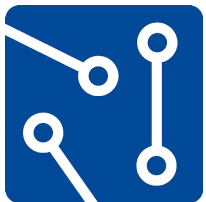




Enhanced Dynamic Compaction

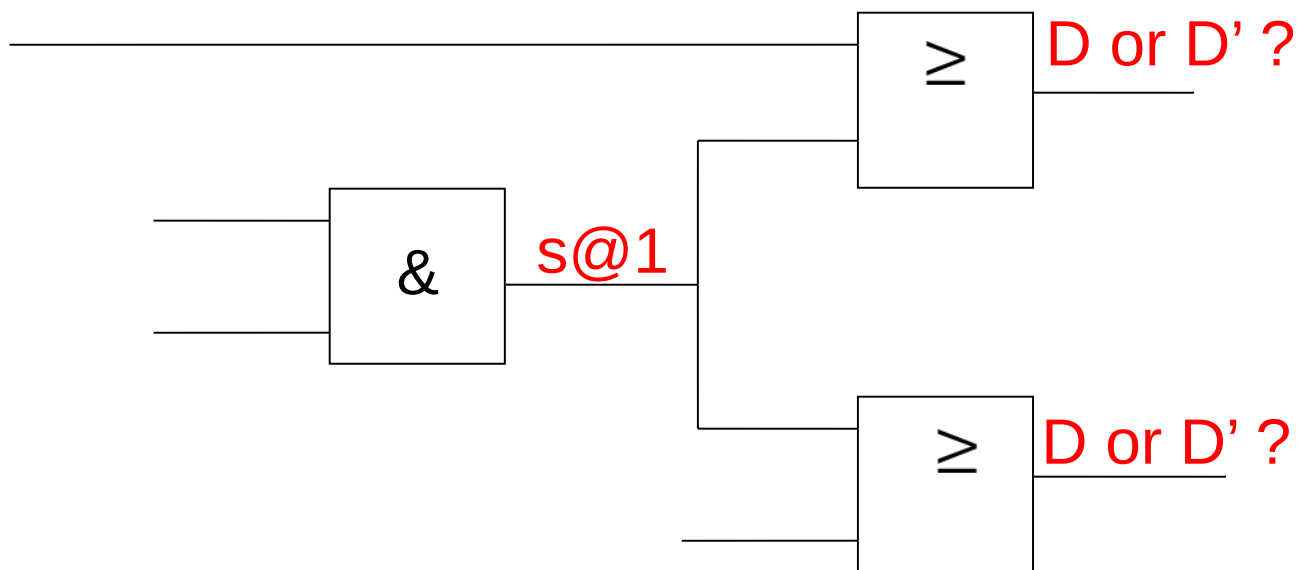
- Improvement 1:
 - Require **internal test conditions**
 - Allows to use 0X0X (e.g. 0100) as a test
 - Still not compatible with test 1100

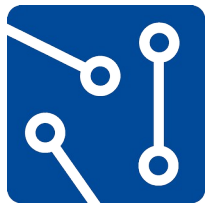




Enhanced Dynamic Compaction

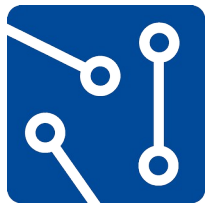
- Improvement 2:
 - Require **fault detection test conditions**
 - Allows to use all tests as a test
 - Encoding in the **SAT-based** ATPG part





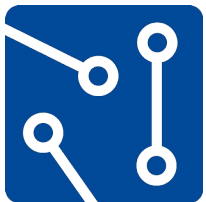
Test Pattern Relaxation

- **Goal:** minimize the **number of specified inputs** in a test t after detecting a fault f and before selecting the next (secondary) target fault.
- Greedy Static Procedure:
 - For every input i which is specified under t and was unspecified before f was targeted:
 - ↪ Unspecify i . If f is not detected by the resulting test, restore the previous value of i .

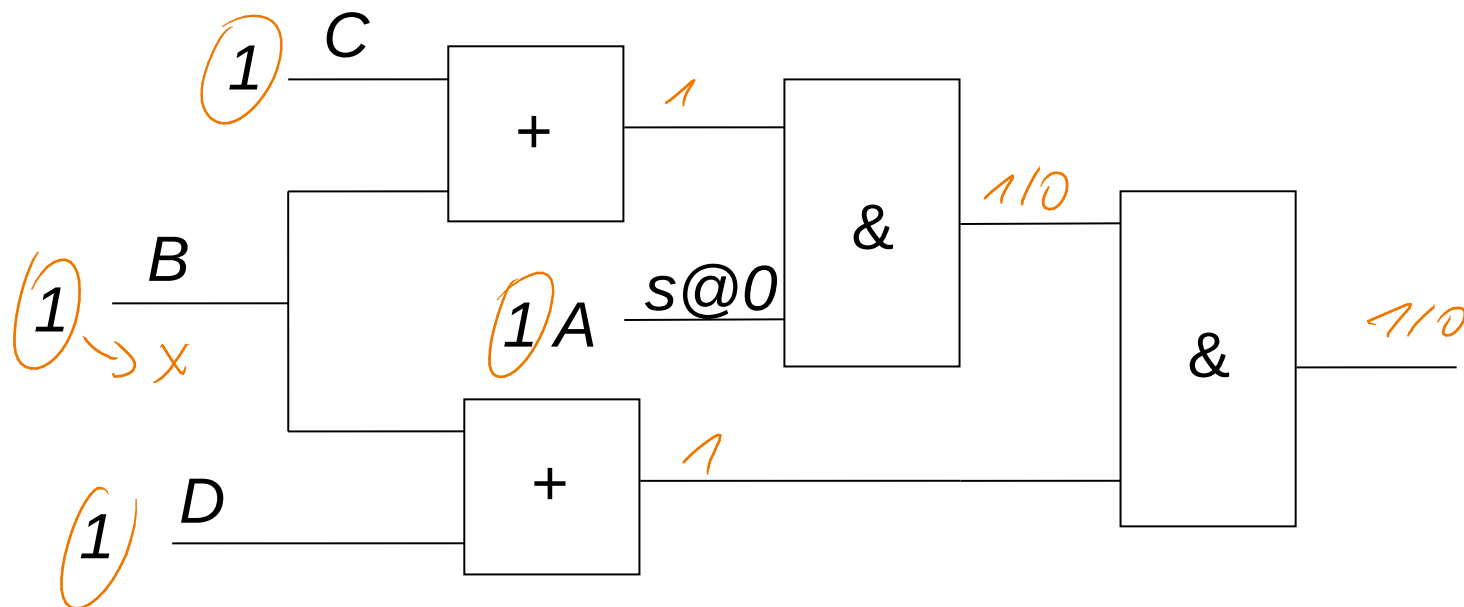


Example of Static Test Pattern Relaxation

- Before targeting f , $t = \underline{\text{xxxx01}}$
- After targeting f , $t = \underline{\text{001x01}}$
- Check if $t = \text{x01x01}$ detects f . If it does not, restore $t = \text{001x01}$. Suppose that $t = \text{x01x01}$ detects f .
- Check if xx1x01 detects f . If it does not, restore $t = \text{x01x01}$. Suppose that xx1x01 does not detect f .
- Check if x0xx01 detects f . If it does not, restore x01x01 . Suppose that $t = \text{x0xx01}$ detects f . This is the final test t .
- t is guaranteed to detect f and all the faults detected by t before f was considered.

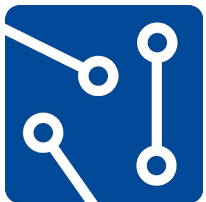


Example 2: Static Test Pattern Relaxation



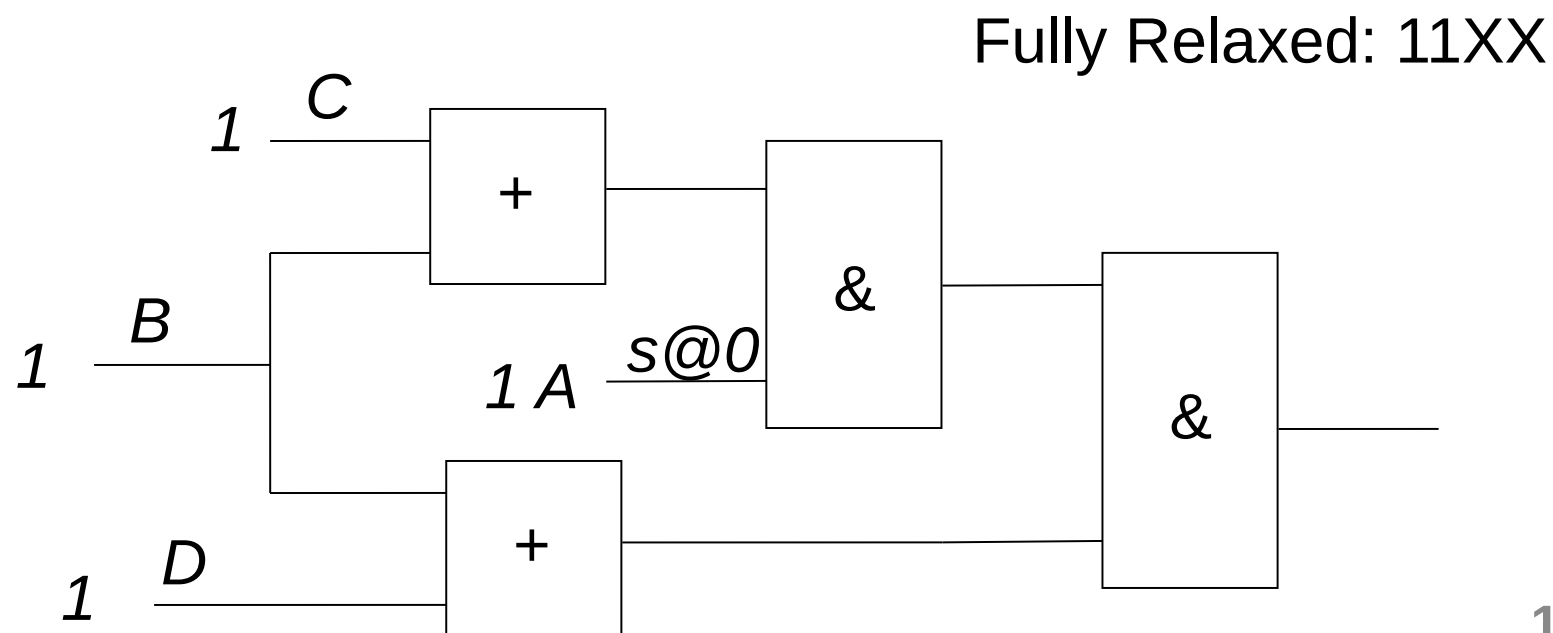
For A stuck-at 0, $t = 1111$

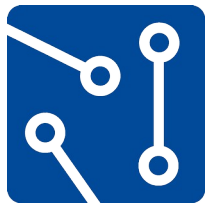
Relaxed: 1X11



Improved Test Pattern Relaxation

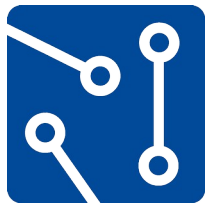
- Targeting of internal circuit values instead of inputs
 - Leads to **dynamic test pattern relaxation**
 - Likewise arguments to compaction
 - More formulation details on SAT-based ATPG





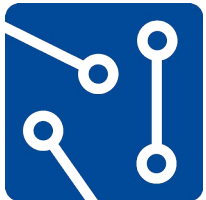
Test Power Consumption

- Application of test patterns requires power, especially if compacted!
- Problem:
 - **Power usage** during test is higher than in functional mode
 - Leads to failing circuits **in test mode only**
- Suboptimal solutions:
 - Size **power delivery network** accordingly
 - ↘ Waste of resources
 - **Reduce clock frequency** during test
 - ↘ Slower test execution, limits delay fault detection



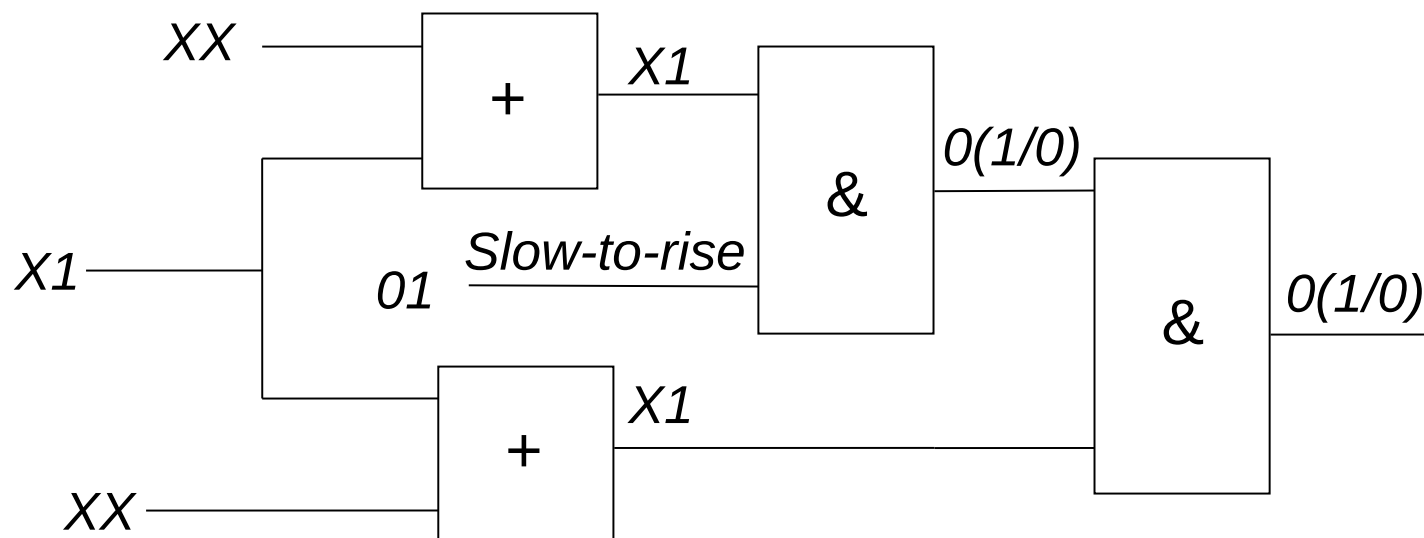
Test Energy Consumption

- Desired solution:
 - Test patterns that do not consume more than **functional** energy
- How to measure energy consumption:
 - **Static power**: Leakage – Pattern independent
 - **Dynamic power**: Switching of a line – Pattern dependent
- Two main power issues:
 - Power consumption during scan
 - ↗ 10101010101010
 - ↘ 11111111111111
 - Power consumption during pattern application



Power-aware Test Pattern Refilling

- Given an incompletely specified test
 - Specify X-values such that power consumption is reduced



- Can also be formulated as an **optimization** problem:
 - Find a test such that the **switchings** are minimized!